

RadML: Complete Manual for Radiation-Tolerant Machine Learning Framework

Rishab Nuguru
Space Labs AI

April 7, 2026

“The core of sustainability is building for the next generation.”

Abstract

Deployment of commercial off-the-shelf (COTS) AI accelerators in deep space is currently limited by their susceptibility to single-event effects (SEE), particularly in deep-nanometer ($< 7\text{nm}$) process nodes where quantum tunneling and confinement effects invalidate classical error rate models. Existing mitigation strategies rely on either computationally expensive offline Monte Carlo simulations (e.g., Geant4) or reactive, static redundancy (e.g., Triple Modular Redundancy), which imposes fixed overheads regardless of environmental severity.

Space Labs AI’s manual documents **RadML**, a **software-defined approach** to radiation tolerance that embeds a physics engine directly into flight software. Designed to work *alongside* traditional radiation-hardened (rad-hard) components and physical shielding, RadML provides an additional layer of protection through intelligent, adaptive fault tolerance. By treating software protection as a dynamic resource allocated according to real-time physical risk, RadML enables the confident utilization of high-performance COTS hardware in hybrid rad-hard architectures.

Core Innovations:

- **Quantum-Corrected SEU Model:** Augments empirical Weibull/Bendel cross-section fits with real-time solutions to the Dirac and Bethe-Salpeter equations, correcting Linear Energy Transfer (LET) calculations for quantum tunneling probabilities in nanoscale transistors.
- **Closed-Loop Adaptive Protection:** Ingests orbital telemetry (particle flux, solar modulation, SAA position) to dynamically scale error correction schemes—shifting from low-overhead Parity checks in quiet regimes to Reed-Solomon (RS-16) coding during Solar Particle Events (SPE).
- **Zero-Overhead Abstractions:** Shifts the computational burden of error correction from runtime execution to compile-time template metaprogramming, achieving near-native performance while providing defense-in-depth reliability.

Application: RadML is designed as a **complementary layer** within hybrid radiation hardening strategies—not a replacement for physical shielding or rad-hard components, but a software-defined enhancement that provides additional safety margins. By layering adaptive, physics-aware fault tolerance atop traditional protection methods, RadML enables mission architects to confidently deploy COTS components in Geostationary Orbit (GEO), Lunar, and interplanetary missions while maintaining the reliability standards required for autonomous deep-space AI systems.

Keywords: Software-Defined Radiation Tolerance, Radiation Hardening by Software (RHBS), Hybrid Rad-Hard Architectures, Quantum-Corrected SEU, Adaptive Fault Tolerance, COTS in Space, Defense-in-Depth.

Contents

1	Fundamental Physics	7
1.1	Single Event Upset (SEU) Rate Calculation	7
1.1.1	Master Equation	7
1.1.2	Integration Explained	7
1.2	Weibull Cross-Section Model (Heavy Ions)	8
1.2.1	Formula	8
1.2.2	Physical Logic	8
1.2.3	Calculation Example	8
1.3	Bendel Proton Model	9
1.3.1	Formula	9
1.3.2	Physical Mechanism	9
1.3.3	Calculation Example	9
2	Advanced Quantum Physics Models	10
2.1	Dirac Equation Solver	10
2.2	Bethe-Salpeter Equation Solver	11
2.3	Green's Function Propagator	12
2.4	Quantum Field Theory Framework	12
2.4.1	QFT Parameters	12
2.4.2	Defect Distribution	13
2.4.3	Quantum Field Class	13
2.5	Quantum-Enhanced Radiation Effects	13
2.6	Crystal Lattice Properties	14
2.6.1	CrystalLattice Structure	14
2.6.2	Validated Material Properties (DFT-based)	14
2.6.3	Semiconductor Properties	14
2.7	Stochastic Defect Evolution	15
2.8	Boltzmann Transport Equation	15
2.9	Quantum Decoherence and Dissipation	16
3	Error Correcting Codes	18
3.1	Hamming(7,4) Code	18
3.1.1	Encoding	18
3.1.2	Syndrome Calculation	18
3.1.3	Example	18
3.2	Reed-Solomon Codes	19
3.2.1	Basic Structure	19
3.2.2	RadML Configurations	19
3.3	Galois Field GF(256) Arithmetic	19
3.3.1	Field Definition	19
3.3.2	Addition (XOR)	20
3.3.3	Multiplication	20
3.3.4	Primitive Element	20
3.3.5	Logarithm Tables	20
3.4	Reed-Solomon Encoding	21
3.4.1	Systematic Encoding	21
3.4.2	Concrete Example	21
3.5	Reed-Solomon Syndrome Calculation	22
3.5.1	Error Polynomial	22
3.5.2	Error Polynomial Structure	22

3.5.3	Example	22
4	Decoding Algorithms	23
4.1	Peterson Decoder (1 Error)	23
4.1.1	Single Error Formulation	23
4.1.2	Algorithm	23
4.1.3	Example	23
4.2	Brute-Force 2-Error Decoder	24
4.2.1	System of Equations	24
4.2.2	Cramer's Rule Solution	24
4.2.3	Algorithm	24
4.2.4	Example	24
4.3	Brute-Force 3-Error Decoder	25
4.3.1	System of Equations	25
4.3.2	Determinant via Cofactor Expansion	25
4.3.3	Cramer's Rule	25
4.3.4	Algorithm	26
4.4	Berlekamp-Massey Algorithm	26
4.4.1	Why This Works	26
4.4.2	Error Locator Polynomial	26
4.4.3	Key Insight	26
4.4.4	Algorithm	27
4.4.5	After Finding $\Lambda(x)$	27
5	Adaptive Protection Logic	28
5.1	Protection Level Selection	28
5.1.1	Decision Function	28
5.1.2	Logic Flow	28
5.2	Scrubbing Interval Calculation	28
5.2.1	Basic Formula	28
5.2.2	Example	28
5.3	MBU-Corrected Scrubbing	29
5.3.1	Refined Formula	29
5.3.2	Physical Meaning	29
5.3.3	Example with MBU Correction	29
5.4	Uncorrectable Error Probability	29
5.4.1	Poisson Model	29
5.4.2	Example	30
6	Orbital Radiation Environment	31
6.1	Geomagnetic Cutoff Rigidity	31
6.1.1	Störmer Formula	31
6.1.2	Rigidity to Energy Conversion	31
6.1.3	Example: $R_c = 1$ GV for protons	31
6.1.4	Cutoff at Equator	31
6.2	McIlwain L-Shell Parameter	32
6.2.1	Definition	32
6.2.2	Example: ISS Orbit	32
6.2.3	Trapped Particle Flux	32
6.3	South Atlantic Anomaly (SAA)	32
6.3.1	Geographic Location	32
6.3.2	Enhanced Flux	32

6.3.3	ISS SAA Statistics	33
6.3.4	Ground Track Calculation	33
6.4	Galactic Cosmic Ray (GCR) Spectrum	33
6.4.1	Adams LET-Based Model (Implemented)	33
6.4.2	Solar Modulation	33
7	Integrated SEU Rate Calculation	34
7.1	Complete Calculation	34
7.1.1	Full Formula	34
7.1.2	Step-by-Step for LEO (ISS)	34
7.2	Mission-Specific SEU Rates	35
8	Memory Overhead and Performance	36
8.1	Memory Layout	36
8.1.1	Single Float with RS-16	36
8.1.2	Cache Line Impact	36
8.2	Computational Overhead	36
8.2.1	For 1M Weight Neural Network	36
8.2.2	Scrubbing Overhead	36
8.3	Block Encoding for Large Data	36
8.3.1	RS(255, k) Constraint	36
8.3.2	Large Layer Encoding	37
8.3.3	Block Interleaving Strategy	37
9	Implementation Details	38
9.1	Type Safety with Templates	38
9.1.1	C++ Template Example	38
9.1.2	Compile-Time Checks	38
9.1.3	Type Aliases	38
9.2	Zero-Cost Abstraction	38
9.2.1	RAII Pattern	38
9.2.2	Zero Runtime Cost	39
9.3	Syndrome Verification Logic	39
9.3.1	Implementation	39
9.3.2	Mathematical Guarantee	39
10	Validation Methodology	40
10.1	Monte Carlo Simulation	40
10.1.1	Test Structure	40
10.1.2	Success Criterion	40
10.2	Error Injection Patterns	40
10.2.1	Random Errors	40
10.2.2	Burst Errors	40
10.2.3	MBU Realistic Patterns (Future Work)	40
11	Summary Tables	41
11.1	Complete Symbol Reference	41
11.2	Algorithm Complexity Summary	41
11.3	Protection Configuration Summary	41

12 Code Implementation Cross-Reference	42
12.1 Galois Field Arithmetic Implementation	42
12.1.1 Field Definition and Primitive Polynomial	42
12.1.2 Addition in GF(256)	42
12.1.3 Multiplication Using Log Tables	43
12.1.4 Table Initialization	43
12.2 Hamming(7,4) Code Implementation	43
12.2.1 Encoding	43
12.2.2 Syndrome Calculation and Error Correction	44
12.3 Weibull Cross-Section Model Implementation	45
12.4 Bendel Proton Model Implementation	46
12.5 Reed-Solomon Encoding Implementation	46
12.5.1 Generator Polynomial Construction	46
12.5.2 Systematic Encoding	47
12.6 Berlekamp-Massey Algorithm Implementation	48
12.7 Brute-Force 2-Error Decoder Implementation	49
12.8 Adaptive Protection Level Selection	50
12.9 South Atlantic Anomaly Model	51
12.10 Scrubbing Interval Calculation	51
12.11 Complete Decoding Pipeline Example	52
12.12 Summary: Math-to-Code Translation Principles	53
13 Template Metaprogramming and Zero-Overhead Abstractions	54
13.1 The Overhead Challenge	54
13.2 Compile-Time vs. Runtime: Key Distinction	54
13.3 Template Parameters as Compile-Time Constants	54
13.3.1 Galois Field: Compile-Time Field Selection	54
13.3.2 Reed-Solomon: Compile-Time ECC Configuration	55
13.4 constexpr: Compile-Time Function Evaluation	56
13.4.1 Compile-Time Table Generation	56
13.5 Type Traits and SFINAE for Zero-Cost Type Safety	57
13.5.1 Compile-Time Type Checking	57
13.5.2 Conditional Type Selection	57
13.6 Template Specialization for Algorithm Selection	57
13.7 Inline and Link-Time Optimization	58
13.7.1 Forced Inlining for Hot Paths	58
13.8 Memory Layout Optimization	59
13.8.1 Compile-Time Size Calculation	59
13.8.2 Cache-Friendly Data Layout	59
13.9 Overhead Comparison: Templates vs. Runtime Polymorphism	60
13.10 Practical Example: Complete Protection Pipeline	60
13.11 Summary: Template Metaprogramming Benefits	60
14 Framework Architecture Diagrams	62
14.1 High-Level System Architecture	62
14.2 Protection Pipeline Flow	62
14.3 Protection Level Selection	62
14.4 Reed-Solomon Pipeline	62
14.5 GF(256) Multiplication	62
14.6 TMR Voting	62
14.7 Memory Layout with ECC	62
14.8 Radiation Environment Mapping	62

15 Defense in Depth: Complete Protection Stack	63
15.1 Defense Layers Overview	63
15.2 Layer 1: Radiation-Aware Memory Placement	63
15.3 Layer 2: Proactive Memory Scrubbing	63
15.4 Layer 3: Error-Correcting Codes (ECC)	64
15.5 Layer 4: Spatial Redundancy (TMR Variants)	64
15.6 Layer 5: Temporal Redundancy	65
15.7 Layer 6: Hybrid Redundancy	65
15.8 Layer 7: Checkpoint and Recovery	65
15.9 Layer 8: Runtime Error Tracking	66
15.10 Layer 9: Power-Aware Protection	66
15.11 Layer 10: Selective Hardening	66
15.12 Defense-in-Depth Integration Diagram	67
15.13 How Layers Work Together: Example Scenario	67
15.14 Physics-Driven Protection Integration	67
15.15 Validation Through Fault Injection	68
15.16 Summary: Defense-in-Depth Effectiveness	68
16 Why RadML Works: Integration Architecture	68
16.1 The Core Integration Pipeline	69
16.2 Why Physics-Driven Protection Works	69
16.2.1 The Problem with Static Protection	69
16.2.2 How RadML Solves This	69
16.3 Why Sensitivity-Based Resource Allocation Works	70
16.4 Why Quantum Corrections Improve Accuracy	70
16.5 Why the Defense-in-Depth Stack Works	71
16.6 Why TMR Variants Exist	71
16.7 Integration Code Path Example	71
16.8 Performance vs Protection Trade-off	72
16.9 Why the Transport Equation Matters	73
16.10 Summary: The RadML Design Philosophy	73
17 Complete RadML Framework Architecture	73
17.1 Framework Overview	74
17.2 Advanced Protection Mechanisms	74
17.2.1 Algorithmic Diversity	74
17.2.2 Neural Network-Based Error Prediction	74
17.3 Research and Architecture Optimization	75
17.3.1 Automated Architecture Search	75
17.3.2 Radiation-Tolerant Variational Autoencoder	75
17.3.3 Radiation-Aware Training	76
17.4 PyTorch Integration	76
17.5 AI-Native Database	77
17.6 Mission Simulation Infrastructure	77
17.6.1 Full Mission Simulator	77
17.6.2 Physics-Based Radiation Simulator	78
17.7 NASA/ESA Validation Protocol	78
17.8 Material Database	79
17.9 Unified API	79
17.10 Framework Component Summary	80
References	81

1 Fundamental Physics

Physics Model Integration

RadML uses a hybrid approach: validated engineering models for production SEU rates, with quantum corrections for improved accuracy.

Production Layer: (physics/radiation_physics.hpp)

- **Weibull/Bendel** cross-sections – Calibrated to ground test data
- **Radiation flux models** – Adams GCR (LET-based), AP-8-inspired trapped protons, AE-8-inspired trapped electrons, SAA

Correction Layer: (quantum_enhanced_radiation.hpp)

- Quantum corrections via `applyQuantumFieldCorrections()` (free function in `quantum_field_theory.hpp`)
- Device sensitivity: SRAM 1.0×, DRAM 1.5×, MRAM 0.1×

Research Layer: (quantum_field_theory.hpp)

- Dirac, Bethe-Salpeter, Green's Functions – for validation

Transport Foundation: (transport_equation.hpp)

- NASA flux = pre-computed transport solutions
- SEU Rate = $\int \Phi(E) \cdot \sigma(E) dE$

Summary: Engineering models → Base SEU rate. Quantum models → Correction factors. Both implemented; quantum layer is optional.

1.1 Single Event Upset (SEU) Rate Calculation

1.1.1 Master Equation

The fundamental SEU rate is computed by integrating the particle flux spectrum with the device cross-section:

$$R_{\text{SEU}} = \int_0^{\infty} \Phi(E) \cdot \sigma(E) dE \quad (1)$$

Table 1: SEU Rate Equation Field Definitions

Symbol	Name	Units	Typical Values
R_{SEU}	SEU Rate	errors/bit/day	10^{-5} to 10^{-3}
$\Phi(E)$	Differential Flux	particles/(cm ² ·s·MeV)	Varies by orbit
$\sigma(E)$	Cross-section	cm ² /bit	10^{-14} to 10^{-8}
E	Particle Energy	MeV	1–10,000

Physical Logic:

$$\boxed{\text{Upset Rate} = (\text{Particles arriving}) \times (\text{Upset probability per particle})} \quad (2)$$

1.1.2 Integration Explained

The integral sums over all energies from 0 to infinity. At each energy E :

- Count particles arriving: $\Phi(E)$
- Multiply by upset chance: $\sigma(E)$
- Add to total rate

Numerical integration using trapezoidal rule:

$$\int_a^b f(E) dE \approx \sum_{i=1}^N \frac{f(E_i) + f(E_{i+1})}{2} \cdot (E_{i+1} - E_i) \quad (3)$$

1.2 Weibull Cross-Section Model (Heavy Ions)

1.2.1 Formula

For heavy ions striking silicon, the SEU cross-section as a function of Linear Energy Transfer (LET) follows:

$$\sigma(L) = \begin{cases} \sigma_{\text{sat}} \cdot \left[1 - \exp \left(- \left(\frac{L - L_{\text{th}}}{W} \right)^s \right) \right] & \text{if } L > L_{\text{th}} \\ 0 & \text{if } L \leq L_{\text{th}} \end{cases} \quad (4)$$

Table 2: Weibull Model Parameters

Symbol	Name	Units	Typical (65nm)
$\sigma(L)$	SEU Cross-section	cm ² /bit	10 ⁻¹⁴ to 10 ⁻⁸
σ_{sat}	Saturation cross-section	cm ² /bit	10 ⁻⁸
L	Linear Energy Transfer	MeV·cm ² /mg	0.1–100
L_{th}	LET Threshold	MeV·cm ² /mg	0.5
W	Weibull Width	MeV·cm ² /mg	1–10
s	Weibull Shape	dimensionless	1–4

1.2.2 Physical Logic

1. Threshold behavior:

$$L < L_{\text{th}} \implies \sigma = 0 \quad (\text{particle too weak}) \quad (5)$$

2. Exponential rise:

$$L \gg L_{\text{th}} \implies \sigma \rightarrow \sigma_{\text{sat}} \quad (\text{maximum probability}) \quad (6)$$

3. Transition region: Shape controlled by W and s

1.2.3 Calculation Example

Given: $\sigma_{\text{sat}} = 10^{-8}$, $L_{\text{th}} = 0.5$, $W = 3.0$, $s = 2$, $L = 5.0$

$$\text{Step 1: } (L - L_{\text{th}}) = 5.0 - 0.5 = 4.5 \quad (7)$$

$$\text{Step 2: } \frac{L - L_{\text{th}}}{W} = \frac{4.5}{3.0} = 1.5 \quad (8)$$

$$\text{Step 3: } \left(\frac{L - L_{\text{th}}}{W} \right)^s = (1.5)^2 = 2.25 \quad (9)$$

$$\text{Step 4: } \exp(-2.25) = 0.1054 \quad (10)$$

$$\text{Step 5: } 1 - 0.1054 = 0.8946 \quad (11)$$

$$\text{Step 6: } \sigma(5.0) = 10^{-8} \times 0.8946 = 8.946 \times 10^{-9} \text{ cm}^2/\text{bit} \quad (12)$$

1.3 Bendel Proton Model

1.3.1 Formula

Protons cause SEUs through nuclear reactions, not direct ionization:

$$\sigma_p(E) = \begin{cases} \sigma_\infty \cdot \left[1 - \exp\left(-0.18 \left(\sqrt{E} - \sqrt{A}\right)\right)\right]^4 & \text{if } E > A \\ 0 & \text{if } E \leq A \end{cases} \quad (13)$$

Note: The argument is $(\sqrt{E} - \sqrt{A})$, representing excess energy above threshold.

Table 3: Bendel Model Parameters

Symbol	Name	Units	Typical Value
$\sigma_p(E)$	Proton cross-section	cm ² /bit	10 ⁻¹⁵ to 10 ⁻¹⁰
σ_∞	Limiting cross-section	cm ² /bit	10 ⁻¹² to 10 ⁻¹⁰
E	Proton Energy	MeV	10–1000
A	Bendel Threshold	MeV	18 (65nm)

1.3.2 Physical Mechanism

$$\text{Proton} + \text{Si nucleus} \rightarrow \text{Heavy recoil ions} \rightarrow \text{SEU} \quad (14)$$

Why the square root? Nuclear reaction cross-section scales with $\sqrt{E}$ from quantum mechanics:

- Higher energy $\rightarrow$ shorter de Broglie wavelength $\lambda = h/p \propto 1/\sqrt{E}$
- Shorter wavelength $\rightarrow$ better “resolution” for nuclear interactions
- Threshold behavior: $E > A$ required for nuclear reaction to occur

1.3.3 Calculation Example

Given: $\sigma_\infty = 5 \times 10^{-11}$, $A = 18$ MeV, $E = 100$ MeV

$$\text{Step 1: } \sqrt{E} = \sqrt{100} = 10 \quad (15)$$

$$\text{Step 2: } \sqrt{A} = \sqrt{18} = 4.243 \quad (16)$$

$$\text{Step 3: } \sqrt{E} - \sqrt{A} = 10 - 4.243 = 5.757 \quad (17)$$

$$\text{Step 4: } 0.18 \times (\sqrt{E} - \sqrt{A}) = 0.18 \times 5.757 = 1.036 \quad (18)$$

$$\text{Step 5: } \exp(-1.036) = 0.3549 \quad (19)$$

$$\text{Step 6: } 1 - 0.3549 = 0.6451 \quad (20)$$

$$\text{Step 7: } (0.6451)^4 = 0.1732 \quad (21)$$

$$\text{Step 8: } \sigma_p(100) = 5 \times 10^{-11} \times 0.1732 = 8.66 \times 10^{-12} \text{ cm}^2/\text{bit} \quad (22)$$

Code Verification (`radiation_physics.hpp`, `bendel_proton_cross_section()`):

```
double Y = 0.18 * (sqrt_E - sqrt_A); // Matches formula above
```

2 Advanced Quantum Physics Models

A full hierarchy of quantum physics models for accurate radiation effect simulation.

Integration Clarification

How these models are used:

1. **Production SEU rates** (Section 7) use Weibull/Bendel + NASA flux models
2. **Quantum corrections** are applied via the free function `applyQuantumFieldCorrections()` (defined in `quantum_field.theory.hpp`), called internally by `QuantumEnhancedRadiation`
3. **Advanced models** (Dirac, BSE, Green's) are used for:
 - Validation against experimental data (`scientific_validation_test.cpp`)
 - Physics-based correction factors
 - Research and device characterization

The quantum layer is **optional**—production code can use classical models alone, or enable quantum corrections for improved accuracy.

2.1 Dirac Equation Solver

File: `physics/advanced_quantum_models.hpp`

The Dirac equation describes relativistic electrons and is essential for high-energy radiation interactions:

$$(i\hbar\gamma^\mu\partial_\mu - mc)\psi = 0 \quad (23)$$

Implementation:

```
class DiracEquationSolver {
    struct DiracMatrices {
        ComplexMatrix gamma0, gamma1, gamma2, gamma3; // 4x4 matrices
        ComplexMatrix gamma5; // Chirality matrix
    };

    // Solve for given momentum and energy
    ComplexVector solveDiracEquation(
        const Eigen::Vector3d& momentum, double energy, double mass);

    // Calculate relativistic scattering cross-section
    double calculateRelativisticCrossSection(
        double incident_energy, double scattering_angle,
        const MaterialProperties& target_material);

    // Relativistic correction to displacement energy
    double calculateRelativisticDisplacementEnergy(
        double non_relativistic_energy, double electron_energy);
};
```

Physical Constants Used:

- $\hbar c = 197.3269804 \text{ eV}\cdot\text{nm}$
- $m_e c^2 = 510998.95 \text{ eV}$ (electron mass)

2.2 Bethe-Salpeter Equation Solver

Bethe-Salpeter equation describes two-particle bound states, crucial for defect cluster formation:

$$\Gamma(p) = \int K(p, p') \Gamma(p') d^4 p' \quad (24)$$

Implementation:

```
class BetheSalpeterSolver {
    // Solve for two-particle bound states
    BSEVector solveBetheSalpeter(
        const BSEMatrix& kernel,
        double energy_binding,
        const Eigen::Vector3d& total_momentum);

    // Calculate defect cluster binding energy
    double calculateClusterBindingEnergy(
        const std::vector<Eigen::Vector3d>& defect_positions,
        const std::vector<DefectType>& defect_types,
        const CrystalLattice& crystal_lattice,
        const QFTParameters& params);

    // Exciton-like bound states in semiconductors
    std::pair<double, BSEVector> calculateExcitonBinding(
        const Eigen::Vector3d& electron_position,
        const Eigen::Vector3d& hole_position,
        const CrystalLattice& crystal_lattice,
        double temperature);
};
```

Design Rationale: Why Simplified BSE?

The Full BSE Problem:

- 4D momentum integration over p and p'
- Ladder diagram summation (infinite Feynman series)
- Self-consistent iteration (kernel depends on solution)
- Computational cost: $O(N^4)$ to $O(N^6)$ scaling
- Runtime: Hours/days on supercomputers (VASP, BerkeleyGW)

RadML's Simplification:

- Converts BSE to eigenvalue problem: $\Gamma = (E_{\text{bind}} - K)^{-1} \Gamma$
- Uses Eigen's complex eigensolver (milliseconds on flight CPU)
- Bound states identified by eigenvalues closest to zero
- Screened Coulomb kernel captures essential defect interactions

What We Preserve (Physics That Matters for SEU):

- Eigenvalue structure $\rightarrow$ bound state existence
- Coulomb screening $\rightarrow$ defect interaction range
- Cluster binding energy $\rightarrow$ MBU probability

Accuracy Trade-off:

Approach	Precision	Runtime
Full BSE (BerkeleyGW)	~ 1 meV	Hours (HPC)
RadML Simplified BSE	~ 0.1 eV	Milliseconds (flight CPU)

For defect clustering prediction in SEU rates, 0.1 eV precision is sufficient—we need qualitative physics (do defects cluster?), not exact exciton energies.

2.3 Green's Function Propagator

Green's functions solve inhomogeneous differential equations for wave propagation:

$$(\nabla^2 + k^2)G(\vec{r}, \vec{r}') = \delta(\vec{r} - \vec{r}') \quad (25)$$

Implementation:

```
class GreensFunctionPropagator {
    // Solve Helmholtz equation
    Eigen::VectorXcd solveHelmholtzEquation(
        const std::vector<Eigen::Vector3d>& source_positions,
        const Eigen::VectorXcd& source_amplitudes,
        double wave_number,
        const std::vector<Eigen::Vector3d>& field_points);

    // Calculate radiation field from particle trajectory
    Eigen::VectorXcd calculateRadiationField(
        const std::vector<Eigen::Vector3d>& particle_trajectory,
        const Eigen::Vector3d& particle_velocity,
        const std::vector<Eigen::Vector3d>& observation_points,
        double frequency);

    // Propagate defects through material
    std::vector<DefectDistribution> propagateDefects(
        const DefectDistribution& initial,
        const GreensMatrix& kernel,
        const std::vector<double>& time_points);

    // Phonon-mediated defect interaction
    GreensMatrix calculatePhononMediatedInteraction(
        const std::vector<Eigen::Vector3d>& defect_positions,
        const std::function<double(const Eigen::Vector3d)>& phonon_dispersion,
        double temperature);
};
```

2.4 Quantum Field Theory Framework

File: physics/quantum_field_theory.hpp

2.4.1 QFT Parameters

```
struct QFTParameters {
    double hbar = 6.582119569e-16;    // Reduced Planck constant (eV*s)
    double potential_coefficient = 0.5;
    double lattice_spacing = 1.0;    // nm
    double time_step = 1.0e-18;    // s
    double omega = 1.0e15;    // Angular frequency (rad/s)
    int dimensions = 3;

    // Particle-specific masses (kg)
    std::unordered_map<ParticleType, double> masses = {
        {Proton, 1.6726219e-27}, {Electron, 9.1093837e-31},
        {Neutron, 1.6749275e-27}, {HeavyIon, 5.0e-27},
        {Muon, 1.8835316e-28}
    };

    // Coupling constants
    std::unordered_map<ParticleType, double> coupling_constants = {
        {Proton, 0.1}, {Electron, 0.1}, {HeavyIon, 0.15},
        {Muon, 0.12}, {Neutrino, 0.01}
    };
};
```

```

// Configurable dynamics
double phase_evolution_rate = 1.0e15; // Hz
double damping_factor = 0.001;
double temperature_threshold = 150.0; // K
};

```

2.4.2 Defect Distribution

```

struct DefectDistribution {
    std::unordered_map<ParticleType, std::vector<double>> interstitials;
    std::unordered_map<ParticleType, std::vector<double>> vacancies;
    std::unordered_map<ParticleType, std::vector<double>> clusters;
};

```

2.4.3 Quantum Field Class

```

template <int Dimensions = 3>
class QuantumField {
    std::vector<std::complex<double>> field_data_;
    std::vector<int> dimensions_;
    double lattice_spacing_;
    ParticleType particle_type_;

    // Initialize with Gaussian distribution
    void initializeGaussian(double mean, double stddev);

    // Initialize coherent state with Gaussian envelope
    void initializeCoherentState(double amplitude, double phase);

    // Calculate Hamiltonian terms
    RealMatrix calculateKineticTerm() const;
    RealMatrix calculatePotentialTerm(
        const QFTParameters& params,
        std::optional<ParticleType> particle_type = std::nullopt);

    // Total field energy
    double calculateTotalEnergy(const QFTParameters& params);
};

```

2.5 Quantum-Enhanced Radiation Effects

File: physics/quantum_enhanced_radiation.hpp

Bridges QFT to practical SEU calculations with fundamental physical constants:

```

class QuantumEnhancedRadiation {
    // Physical constants
    static constexpr double ELECTRON_CHARGE = 1.602176634e-19; // C
    static constexpr double BOLTZMANN_K = 8.617333262e-5; // eV/K
    static constexpr double HBAR_EV_S = 6.582119569e-16; // eV*s

    // Quantum-corrected charge deposition
    double calculateQuantumChargeDeposition(
        double particle_energy, // MeV
        double let, // MeV*cm^2/mg
        ParticleType particle_type);

    // Temperature-dependent critical charge (quantum tunneling)
    double calculateTemperatureCriticalCharge(
        double base_critical_charge, // fC

```

```

        double temperature);          // K

// Device-specific sensitivity
double calculateDeviceSensitivity(
    MemoryDeviceType device_type,
    double feature_size_nm);

// Enhanced bit flip probability (classical + quantum)
double calculateEnhancedBitFlipProbability(
    double deposited_charge,
    MemoryDeviceType device_type,
    double temperature);

// Apply radiation effects to memory region
uint32_t applyQuantumEnhancedRadiation(
    void* memory, size_t size_bytes,
    double particle_energy, double let,
    ParticleType particle_type,
    MemoryDeviceType device_type,
    uint32_t duration_ms);

// Multi-bit upset size using quantum clustering
uint32_t calculateQuantumMBUSize(
    double deposited_charge,
    ParticleType particle_type); // Returns 1-8 bits
};

```

Memory Device Types Supported:

Enum	Description
SRAM_6T	6-transistor SRAM cell
SRAM_8T	8-transistor SRAM (more robust)
DRAM	Dynamic RAM
FLASH_SLC	Single-level cell Flash
FLASH_MLC	Multi-level cell Flash
MRAM	Magnetoresistive RAM
FRAM	Ferroelectric RAM

2.6 Crystal Lattice Properties

Files: physics/crystal_lattice_properties.hpp, physics/quantum_field_theory.hpp

2.6.1 CrystalLattice Structure

```

struct CrystalLattice {
    enum class Type { FCC, BCC, DIAMOND };

    Type type = DIAMOND;          // Silicon default
    double lattice_constant = 5.43; // Angstrom (Si)
    double barrier_height = 1.0; // eV
};

```

2.6.2 Validated Material Properties (DFT-based)

2.6.3 Semiconductor Properties

Table 4: Crystal Lattice Parameters (Validated)

Property	FCC	BCC	Diamond
Packing Density	0.74 ($\pi/3\sqrt{2}$)	0.68 ($\pi\sqrt{3}/8$)	0.34 ($\pi\sqrt{3}/16$)
Coordination Number	12	8	4
Vacancy Energy (eV)	1.0–2.0	2.0–5.0	5.0–10.0
Interstitial Energy (eV)	1.5–3.0	3.0–6.0	6.0–12.0

```

struct SemiconductorProperties {
    double bandgap_ev = 1.12;           // Silicon at 300K
    double effective_mass_ratio = 0.26; // m*/m_e
    double dielectric_constant = 11.7;  // Si relative permittivity
    double lattice_constant_nm = 0.543; // Si
    double critical_charge_fc = 15.0;   // Critical charge (fC)
    double temperature_k = 300.0;      // Operating temperature
};

```

2.7 Stochastic Defect Evolution

File: physics/stochastic_models.hpp

Solves stochastic differential equations for defect concentration evolution:

$$dC = \underbrace{\mu(C, t) dt}_{\text{drift}} + \underbrace{\sigma(C, t) dW}_{\text{diffusion}} \quad (26)$$

```

struct MaterialParameters {
    double diffusion_coefficient; // m^2/s
    double recombination_radius; // Angstrom
    double migration_energy;     // eV
    double displacement_energy;  // eV
};

SimulationResults solveStochasticDE(
    const Eigen::VectorXd& initial_concentrations,
    const DriftFunction& drift_term,
    const DiffusionFunction& diffusion_term,
    int time_steps,
    double simulation_time,
    double temperature,
    double applied_stress);

```

2.8 Boltzmann Transport Equation

File: physics/transport_equation.hpp

The Boltzmann transport equation describes particle fluence $\Phi(\vec{r}, \hat{\Omega}, E, t)$ through materials:

$$\frac{\partial \Phi}{\partial t} + \hat{\Omega} \cdot \nabla \Phi + \Sigma_t \Phi = \int \Sigma_s(\hat{\Omega}' \rightarrow \hat{\Omega}, E' \rightarrow E) \Phi(\hat{\Omega}', E') d\Omega' dE' + S \quad (27)$$

where Σ_t is the total cross-section, Σ_s is the scattering kernel, and S is the source term.

Interface:

```

struct TransportSolution {
    Eigen::Tensor<double, 3> fluence; // Phi(x, Omega, E)
    double convergence_error;
};

```

```
// Setup radiation source from environment
Eigen::Tensor<double, 3> setupRadiationSource(
    const RadiationEnvironment& env,
    int spatial_points, int angular_points, int energy_bins,
    ParticleType particle_type);

// Generate material cross-sections
Eigen::Tensor<double, 2> generateMaterialCrossSections(
    const MaterialProperties& material, int energy_bins,
    ParticleType particle_type);

// Solve transport equation
TransportSolution solveTransportEquation(
    const Eigen::Tensor<double, 3>& initial_fluence,
    const Eigen::Tensor<double, 3>& source,
    const Eigen::Tensor<double, 2>& sigma_t,
    const Eigen::Tensor<double, 4>& sigma_s);

// Calculate dose from fluence
Eigen::Tensor<double, 1> calculateDoseDistribution(
    const Eigen::Tensor<double, 3>& fluence,
    double material_density);
```

Relationship to SEU Calculations:

Transport → SEU Connection

The transport equation provides the **theoretical foundation** for the production SEU calculations:

1. **Transport equation** → Fluence $\Phi(E)$ at device location
2. **NASA flux models** (GCR, AP-8) are **pre-computed solutions** to transport through Earth's magnetosphere
3. **Weibull/Bendel** cross-sections $\sigma(E)$ are **empirically fitted** device responses
4. **SEU Rate** = $\int \Phi(E) \cdot \sigma(E) dE$ combines both

The transport solver enables custom shielding analysis; the NASA models provide validated magnetospheric transport without re-solving the equation.

2.9 Quantum Decoherence and Dissipation

File: physics/quantum_models.hpp

Extended QFT parameters for open quantum systems:

```
struct ExtendedQFTParameters : public QFTParameters {
    std::unordered_map<ParticleType, double> decoherence_rates;
    std::unordered_map<ParticleType, double> dissipation_coefficients;

    // Default: 1e-12 for decoherence, 0.01 for dissipation
};

// Calculate quantum decoherence effects
double calculateQuantumDecoherence(
    const DefectDistribution& defects,
    double temperature,
    const ExtendedQFTParameters& params,
    ParticleType particle_type);

// Calculate quantum transition probability
```



```
double calculateQuantumTransitionProbability(  
    double incident_energy,  
    double temperature,  
    const QFTParameters& params,  
    ParticleType particle_type);
```

3 Error Correcting Codes

3.1 Hamming(7,4) Code

3.1.1 Encoding

The Hamming(7,4) code takes 4 data bits and adds 3 parity bits:

$$\begin{bmatrix} p_1 \\ p_2 \\ d_1 \\ p_3 \\ d_2 \\ d_3 \\ d_4 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} d_1 \\ d_2 \\ d_3 \\ d_4 \end{bmatrix} \quad (28)$$

Parity bit equations (using XOR $\oplus$ as addition in GF(2)):

$$p_1 = d_1 \oplus d_2 \oplus d_4 \quad (29)$$

$$p_2 = d_1 \oplus d_3 \oplus d_4 \quad (30)$$

$$p_3 = d_2 \oplus d_3 \oplus d_4 \quad (31)$$

Table 5: Hamming Code Field Definitions

Symbol	Name	Meaning
d_i	Data bit	Original information bit ($i = 1, 2, 3, 4$)
p_j	Parity bit	Redundancy for error detection ($j = 1, 2, 3$)
$\oplus$	XOR operation	Addition in GF(2)

3.1.2 Syndrome Calculation

For received word $\mathbf{r} = [r_1, r_2, r_3, r_4, r_5, r_6, r_7]$:

$$s_1 = r_1 \oplus r_3 \oplus r_5 \oplus r_7 \quad (32)$$

$$s_2 = r_2 \oplus r_3 \oplus r_6 \oplus r_7 \quad (33)$$

$$s_3 = r_4 \oplus r_5 \oplus r_6 \oplus r_7 \quad (34)$$

Syndrome to error position:

$$S = [s_1, s_2, s_3]_2 = \text{binary representation of error position} \quad (35)$$

3.1.3 Example

Data: $[d_1, d_2, d_3, d_4] = [1, 0, 1, 1]$

$$p_1 = 1 \oplus 0 \oplus 1 = 0 \quad (36)$$

$$p_2 = 1 \oplus 1 \oplus 1 = 1 \quad (37)$$

$$p_3 = 0 \oplus 1 \oplus 1 = 0 \quad (38)$$

Table 6: Hamming Syndrome Decoding Table

Syndrome S	Error Position
000	No error
001	Position 4 (p_3)
010	Position 2 (p_2)
011	Position 6 (d_3)
100	Position 1 (p_1)
101	Position 5 (d_2)
110	Position 3 (d_1)
111	Position 7 (d_4)

Codeword: $[0, 1, 1, 0, 0, 1, 1]$

Error injection: Flip bit 5 (position of d_2)

Received: $[0, 1, 1, 0, 1, 1, 1]$ ($0 \rightarrow 1$ at position 5)

$$s_1 = 0 \oplus 1 \oplus 1 \oplus 1 = 1 \quad (39)$$

$$s_2 = 1 \oplus 1 \oplus 1 \oplus 1 = 0 \quad (40)$$

$$s_3 = 0 \oplus 1 \oplus 1 \oplus 1 = 1 \quad (41)$$

Syndrome: $S = [1, 0, 1]_2 = 5_{10} \rightarrow$ Error at position 5 ✓

Correction: Flip bit at position 5: $r'_5 = r_5 \oplus 1 = 1 \oplus 1 = 0$

3.2 Reed-Solomon Codes

3.2.1 Basic Structure

Reed-Solomon codes are denoted as $RS(n, k)$ over $GF(2^m)$:

Table 7: Reed-Solomon Parameters

Parameter	Name	Meaning
n	Block length	Total symbols in codeword
k	Message length	Number of data symbols
$n - k$	Redundancy	Number of ECC symbols
t	Error correction capability	$t = \frac{n-k}{2}$ symbols
m	Symbol size	Bits per symbol (typically $m = 8$)

For $GF(256)$ with $m = 8$:

$$n_{\max} = 2^m - 1 = 255 \text{ symbols} \quad (42)$$

3.2.2 RadML Configurations

3.3 Galois Field $GF(256)$ Arithmetic

3.3.1 Field Definition

$$GF(256) = \{0, 1, 2, \dots, 255\} \quad (43)$$

Important: Operations are NOT regular arithmetic!

Table 8: RadML Reed-Solomon Configurations

Config	t	$n - k$	Data (bytes)	ECC (bytes)	Overhead
RS Light	2	4	4 (float)	4	100%
RS Standard	3	6	4 (float)	6	150%
RS Heavy	4	8	4 (float)	8	200%

3.3.2 Addition (XOR)

$$a \oplus b = a \text{ XOR } b \quad (44)$$

Example:

$$163 \oplus 91 = 10100011_2 \oplus 01011011_2 = 11111000_2 = 248 \quad (45)$$

Properties:

$$a \oplus a = 0 \quad (46)$$

$$a \oplus 0 = a \quad (47)$$

$$a \oplus b = b \oplus a \quad (\text{commutative}) \quad (48)$$

$$-a = a \quad (\text{self-inverse}) \quad (49)$$

3.3.3 Multiplication

Defined via **primitive polynomial** $p(x) = x^8 + x^4 + x^3 + x^2 + 1$ (hex: 0x11D):

$$a \cdot b = (a \times b) \mod p(x) \quad (50)$$

Example: $3 \cdot 5$ in $\text{GF}(256)$

$$3 \cdot 5 = (11_2) \cdot (101_2) \quad (51)$$

$$= 11_2 \times 101_2 = 1111_2 = 15_{10} \quad (52)$$

$$\text{Since } 15 < 256, \text{ no reduction needed} \quad (53)$$

$$= 15 \quad (54)$$

3.3.4 Primitive Element

A **primitive element** α satisfies:

$$\{\alpha^0, \alpha^1, \alpha^2, \dots, \alpha^{254}\} = \text{GF}(256) \setminus \{0\} \quad (55)$$

For primitive polynomial 0x11D: $\alpha = 2$

3.3.5 Logarithm Tables

$$\log_\alpha(x) = i \text{ such that } \alpha^i = x \quad (56)$$

$$\exp_\alpha(i) = \alpha^i \quad (57)$$

Fast multiplication using logs:

$$a \cdot b = \alpha^{(\log_\alpha(a) + \log_\alpha(b)) \mod 255} \quad (58)$$

Example: $7 \cdot 11$ in $\text{GF}(256)$

$$\log_{\alpha}(7) = 112 \quad (59)$$

$$\log_{\alpha}(11) = 109 \quad (60)$$

$$112 + 109 = 221 \quad (61)$$

$$\alpha^{221} = 77 \quad (62)$$

$$\therefore 7 \cdot 11 = 77 \quad (63)$$

3.4 Reed-Solomon Encoding

3.4.1 Systematic Encoding

Message polynomial:

$$m(x) = m_0 + m_1x + \dots + m_{k-1}x^{k-1} \quad (64)$$

Generator polynomial:

$$g(x) = \prod_{i=0}^{2t-1} (x - \alpha^i) \quad (65)$$

Encoding steps:

1. Shift message: $x^{n-k} \cdot m(x)$
2. Compute remainder: $r(x) = [x^{n-k} \cdot m(x)] \bmod g(x)$
3. Codeword: $c(x) = x^{n-k} \cdot m(x) - r(x)$

Result:

$$c(x) = \underbrace{m_0, m_1, \dots, m_{k-1}}_{\text{data symbols}} \mid \underbrace{r_0, r_1, \dots, r_{n-k-1}}_{\text{ECC symbols}} \quad (66)$$

3.4.2 Concrete Example

RS(7, 3) over GF(8) with $t = 2$

Message: $m = [5, 2, 6]$ (3 data symbols)

Generator polynomial:

$$g(x) = (x - \alpha^0)(x - \alpha^1)(x - \alpha^2)(x - \alpha^3) \quad (67)$$

$$= (x - 1)(x - \alpha)(x - \alpha^2)(x - \alpha^3) \quad (68)$$

After polynomial multiplication in GF(8):

$$g(x) = x^4 + 3x^3 + 2x^2 + 4x + 7 \quad (69)$$

Message polynomial: $m(x) = 5 + 2x + 6x^2$

Shift by $n - k = 4$: $x^4m(x) = 5x^4 + 2x^5 + 6x^6$

Compute remainder: Through polynomial long division in GF(8):

$$r(x) = 1 + 4x + 3x^2 + 2x^3 \quad (70)$$

Codeword:

$$c = \underbrace{[5, 2, 6]}_{\text{data}} \underbrace{[1, 4, 3, 2]}_{\text{ECC}} \quad (71)$$

3.5 Reed-Solomon Syndrome Calculation

3.5.1 Error Polynomial

Received word:

$$r(x) = c(x) + e(x) \quad (72)$$

where $e(x)$ is the error polynomial.

Syndrome computation:

$$S_i = r(\alpha^i) = c(\alpha^i) + e(\alpha^i) = e(\alpha^i) \quad \text{for } i = 0, 1, \dots, 2t - 1 \quad (73)$$

Since $c(\alpha^i) = 0$ by construction (α^i are roots of $g(x)$).

Syndrome vector:

$$\mathbf{S} = [S_0, S_1, \dots, S_{2t-1}] \quad (74)$$

Properties:

$$\mathbf{S} = \mathbf{0} \iff \text{No error} \quad (75)$$

$$\mathbf{S} \neq \mathbf{0} \iff \text{Error(s) present} \quad (76)$$

3.5.2 Error Polynomial Structure

$$e(x) = e_{j_1}x^{j_1} + e_{j_2}x^{j_2} + \dots + e_{j_v}x^{j_v} \quad (77)$$

where $v \leq t$ is the number of errors.

Syndrome equations:

$$S_i = \sum_{\ell=1}^v e_{j_\ell} \cdot \alpha^{i \cdot j_\ell} \quad \text{for } i = 0, 1, \dots, 2t - 1 \quad (78)$$

3.5.3 Example

2 errors at positions $j_1 = 3$, $j_2 = 5$ with magnitudes $e_1 = 7$, $e_2 = 11$:

$$S_0 = 7 \cdot \alpha^{0 \cdot 3} + 11 \cdot \alpha^{0 \cdot 5} = 7 \cdot 1 + 11 \cdot 1 = 7 \oplus 11 = 12 \quad (79)$$

$$S_1 = 7 \cdot \alpha^{1 \cdot 3} + 11 \cdot \alpha^{1 \cdot 5} = 7 \cdot \alpha^3 + 11 \cdot \alpha^5 \quad (80)$$

$$S_2 = 7 \cdot \alpha^{2 \cdot 3} + 11 \cdot \alpha^{2 \cdot 5} = 7 \cdot \alpha^6 + 11 \cdot \alpha^{10} \quad (81)$$

$$S_3 = 7 \cdot \alpha^{3 \cdot 3} + 11 \cdot \alpha^{3 \cdot 5} = 7 \cdot \alpha^9 + 11 \cdot \alpha^{15} \quad (82)$$

4 Decoding Algorithms

4.1 Peterson Decoder (1 Error)

4.1.1 Single Error Formulation

For single error:

$$e(x) = e_j x^j \quad (83)$$

Syndrome equations:

$$S_0 = e_j \cdot \alpha^{0 \cdot j} = e_j \quad (84)$$

$$S_1 = e_j \cdot \alpha^{1 \cdot j} = e_j \cdot \alpha^j \quad (85)$$

Error location:

$$X_j = \alpha^j = \frac{S_1}{S_0} \quad (86)$$

Error magnitude:

$$e_j = S_0 \quad (87)$$

4.1.2 Algorithm

Algorithm 1 Peterson Single Error Decoder

```

1: if  $\mathbf{S} = [0, 0, \dots, 0]$  then
2:   return No error
3: end if
4: if Only  $S_0$  and  $S_1$  non-zero then
5:    $X \leftarrow S_1/S_0$ 
6:    $j \leftarrow \log_\alpha(X)$ 
7:    $e \leftarrow S_0$ 
8:   return (position= $j$ , magnitude= $e$ )
9: else
10:  return Multiple errors, use next decoder
11: end if

```

4.1.3 Example

Syndromes (implementation-aligned): $S_1 = 13$, $S_2 = 26$

$$X = \frac{S_2}{S_1} = \frac{26}{13} = 2 = \alpha^1 \quad (88)$$

Exponent index: $k = 1$

In the code path, position is array-indexed:

$$\text{pos} = n - 1 - k \quad (89)$$

For $n = 10$, $\text{pos} = 8$.

Magnitude is computed as:

$$e = S_1 \cdot \alpha^{-k} = \frac{S_1}{\alpha^k} \quad (90)$$

Then the decoder verifies consistency with S_3 (when available) before applying correction.

Complexity: $O(n)$ – just two GF divisions and one log lookup

4.2 Brute-Force 2-Error Decoder

4.2.1 System of Equations

For errors at positions j_1, j_2 with magnitudes e_1, e_2 :

$$S_1 = e_1 X_1 \oplus e_2 X_2 \quad (91)$$

$$S_2 = e_1 X_1^2 \oplus e_2 X_2^2 \quad (92)$$

where $X_i = \alpha^{j_i}$ (error locator values).

Matrix form:

$$\begin{bmatrix} X_1 & X_2 \\ X_1^2 & X_2^2 \end{bmatrix} \begin{bmatrix} e_1 \\ e_2 \end{bmatrix} = \begin{bmatrix} S_1 \\ S_2 \end{bmatrix} \quad (93)$$

4.2.2 Cramer's Rule Solution

Determinant:

$$\det = X_1 \cdot X_2^2 \oplus X_2 \cdot X_1^2 = X_1 X_2 (X_2 \oplus X_1) \quad (94)$$

Solutions:

$$e_1 = \frac{S_1 X_2^2 \oplus S_2 X_2}{\det} \quad (95)$$

$$e_2 = \frac{S_1 X_1^2 \oplus S_2 X_1}{\det} \quad (96)$$

Note: In $\text{GF}(2^m)$, subtraction = addition (both XOR).

4.2.3 Algorithm

Algorithm 2 Brute-Force 2-Error Decoder

```

1: for pos1 ← 0 to n − 1 do
2:   for pos2 ← pos1 + 1 to n − 1 do
3:     X1 ← αpos1
4:     X2 ← αpos2
5:     det ← X1 · X2 · (X1 ⊕ X2)
6:     if det ≠ 0 then
7:       e1 ← (S1 · X22 ⊕ S2 · X2) / det
8:       e2 ← (S1 · X12 ⊕ S2 · X1) / det
9:       if e1 · X1 ⊕ e2 · X2 = S1 and e1 · X12 ⊕ e2 · X22 = S2 then
10:        return [(pos1, e1), (pos2, e2)]
11:      end if
12:    end if
13:  end for
14: end for

```

4.2.4 Example

Given: $n = 10$, $S_1 = 42$, $S_2 = 137$

Try positions (3, 7):

$$X_1 = \alpha^{k_1}, \quad k_1 = n - 1 - p_1 \quad (97)$$

$$X_2 = \alpha^{k_2}, \quad k_2 = n - 1 - p_2 \quad (98)$$

$$X_1^2 = X_1 \cdot X_1, \quad X_2^2 = X_2 \cdot X_2 \quad (99)$$

$$\det = X_1 X_2 (X_2 \oplus X_1) \quad (100)$$

$$e_1 = \frac{S_1 X_2^2 \oplus S_2 X_2}{\det}, \quad e_2 = \frac{S_2 X_1 \oplus S_1 X_1^2}{\det} \quad (101)$$

Implementation note: All products/divisions above are computed in GF(256) using the lookup-table arithmetic in `galois_field.hpp` (`multiply()`, `divide()`), not integer arithmetic.

Verification step (as implemented):

$$\mathbf{r}' = \mathbf{r} \oplus (e_1 \text{ at } p_1, e_2 \text{ at } p_2) \quad (102)$$

$$\mathbf{S}' = \text{rs_calc_syndromes}(\mathbf{r}') \quad (103)$$

$$\mathbf{S}' = \mathbf{0} \Rightarrow \text{candidate accepted} \quad (104)$$

Complexity: $O(n^2)$ – try all position pairs

4.3 Brute-Force 3-Error Decoder

4.3.1 System of Equations

$$\begin{bmatrix} X_1 & X_2 & X_3 \\ X_1^2 & X_2^2 & X_3^2 \\ X_1^3 & X_2^3 & X_3^3 \end{bmatrix} \begin{bmatrix} e_1 \\ e_2 \\ e_3 \end{bmatrix} = \begin{bmatrix} S_1 \\ S_2 \\ S_3 \end{bmatrix} \quad (105)$$

Vandermonde-type matrix

4.3.2 Determinant via Cofactor Expansion

$$\det = X_1 \begin{vmatrix} X_2^2 & X_3^2 \\ X_2^3 & X_3^3 \end{vmatrix} \oplus X_1^2 \begin{vmatrix} X_2 & X_3 \\ X_2^3 & X_3^3 \end{vmatrix} \oplus X_1^3 \begin{vmatrix} X_2 & X_3 \\ X_2^2 & X_3^2 \end{vmatrix} \quad (106)$$

$$= X_1(X_2^2 X_3^3 \oplus X_3^2 X_2^3) \quad (107)$$

$$= X_1(X_2^2 X_3^3 \oplus X_3^2 X_2^3) \quad (108)$$

$$\oplus X_1^2(X_2 X_3^3 \oplus X_3 X_2^3) \quad (109)$$

$$\oplus X_1^3(X_2 X_3^2 \oplus X_3 X_2^2) \quad (110)$$

In GF(2^m): No sign changes (subtraction = addition = XOR)

4.3.3 Cramer's Rule

$$e_1 = \frac{\begin{vmatrix} S_1 & X_2 & X_3 \\ S_2 & X_2^2 & X_3^2 \\ S_3 & X_2^3 & X_3^3 \end{vmatrix}}{\det} \quad (111)$$

Similarly for e_2 and e_3 .

4.3.4 Algorithm

Algorithm 3 Brute-Force 3-Error Decoder

```

1: for pos1 ← 0 to n − 1 do
2:   for pos2 ← pos1 + 1 to n − 1 do
3:     for pos3 ← pos2 + 1 to n − 1 do
4:       X1, X2, X3 ← αpos1, αpos2, αpos3
5:       det ← compute_3x3_determinant(X1, X2, X3)
6:       if det ≠ 0 then
7:         e1 ← compute_3x3_determinant_col1(S1, S2, S3, X2, X3) / det
8:         e2 ← compute_3x3_determinant_col2(X1, S1, S2, S3, X3) / det
9:         e3 ← compute_3x3_determinant_col3(X1, X2, S1, S2, S3) / det
10:        if verify_syndromes(e1, e2, e3, X1, X2, X3, S1, S2, S3) then
11:          return [(pos1, e1), (pos2, e2), (pos3, e3)]
12:        end if
13:      end if
14:    end for
15:  end for
16: end for

```

Complexity: $O(n^3)$ – try all position triplets

4.4 Berlekamp-Massey Algorithm

4.4.1 Why This Works

The syndromes $S_1, S_2, \dots, S_{2t}$ are evaluations of the error polynomial at successive powers of α . Any such sequence has a **minimal polynomial**—the error locator $\Lambda(x)$ —whose roots reveal error positions. The Berlekamp-Massey algorithm finds this minimal polynomial *iteratively* by assuming the simplest $\Lambda(x)$ that fits the syndromes observed so far, then adjusting minimally when discrepancies occur.

Intuition: Think of BM as incrementally building a linear feedback shift register (LFSR) that generates the syndrome sequence. The shortest LFSR that produces the syndromes corresponds to the error locator polynomial. Each iteration either confirms the current LFSR is correct (discrepancy = 0) or updates it to account for new syndrome information.

4.4.2 Error Locator Polynomial

Most general RS decoder for $v \leq t$ errors.

Error Locator Polynomial:

$$\Lambda(x) = \prod_{\ell=1}^v (1 - X_\ell x) = \Lambda_0 + \Lambda_1 x + \dots + \Lambda_v x^v \quad (112)$$

where $X_\ell = \alpha^{j_\ell}$ are error locators.

4.4.3 Key Insight

Syndromes satisfy the linear recurrence:

$$S_{i+v} = - \sum_{k=1}^v \Lambda_k S_{i+v-k} \quad \text{for } i = 0, 1, 2, \dots \quad (113)$$

Note on $\mathbf{GF}(2^m)$: In characteristic-2 fields, $-x = x$ (additive inverse equals the element itself), so the negative sign is mathematically redundant. The formula is often written without it:

$$S_{i+v} = \sum_{k=1}^v \Lambda_k S_{i+v-k} \quad (\text{equivalent in } \mathbf{GF}(2^m)) \quad (114)$$

The negative sign is retained here for generality with other finite fields.

Berlekamp-Massey finds shortest $\Lambda(x)$ satisfying this recurrence.

4.4.4 Algorithm

Algorithm 4 Berlekamp-Massey Algorithm

```

1:  $\Lambda(x) \leftarrow 1$ 
2:  $B(x) \leftarrow 1$ 
3:  $L \leftarrow 0$  ▷ Current length
4:  $m \leftarrow 1$  ▷ Since last length change
5:  $b \leftarrow 1$  ▷ Correction factor
6: for  $r \leftarrow 0$  to  $2t - 1$  do
7: ▷ Compute discrepancy
8:    $\Delta \leftarrow S[r] \oplus \sum_{i=1}^L \Lambda[i] \cdot S[r-i]$ 
9:   if  $\Delta = 0$  then
10:      $m \leftarrow m + 1$ 
11:   else
12:      $T(x) \leftarrow \Lambda(x)$ 
13:      $\Lambda(x) \leftarrow \Lambda(x) \oplus (\Delta/b) \cdot x^m \cdot B(x)$ 
14:     if  $2L \leq r$  then
15:        $L \leftarrow r + 1 - L$ 
16:        $B(x) \leftarrow T(x)$ 
17:        $b \leftarrow \Delta$ 
18:        $m \leftarrow 1$ 
19:     else
20:        $m \leftarrow m + 1$ 
21:     end if
22:   end if
23: end for
24: return  $\Lambda(x)$ 

```

4.4.5 After Finding $\Lambda(x)$

1. **Chien Search:** Find roots X_ℓ by testing $\Lambda(\alpha^{-i})$ for $i = 0$ to $n - 1$
2. **Forney Algorithm:** Compute error magnitudes

$$e_\ell = -\frac{X_\ell \cdot \Omega(X_\ell^{-1})}{\Lambda'(X_\ell^{-1})} \quad (115)$$

where $\Omega(x)$ is the error evaluator polynomial.

Complexity: $O(t^2)$ for finding $\Lambda(x)$, $O(n \cdot t)$ for Chien search

5 Adaptive Protection Logic

5.1 Protection Level Selection

5.1.1 Decision Function

$$\text{ProtectionLevel}(R_{\text{SEU}}) = \begin{cases} \text{VERY_HIGH} & \text{if } R_{\text{SEU}} > 10^{-3} \\ \text{HIGH} & \text{if } 10^{-4} < R_{\text{SEU}} \leq 10^{-3} \\ \text{MODERATE} & \text{if } 10^{-5} < R_{\text{SEU}} \leq 10^{-4} \\ \text{MINIMAL} & \text{if } R_{\text{SEU}} \leq 10^{-5} \end{cases} \quad (116)$$

Table 9: Protection Scheme Mapping

Level	ECC Scheme	Correction	Overhead	When to Use
MINIMAL	Parity	Detection only	12.5%	Deep space, low flux
MODERATE	Hamming(7,4)×2	1 bit/nibble	75%	LEO, nominal
HIGH	RS-8	4 bytes	200%	GEO, moderate
VERY_HIGH	RS-16	8 bytes	400%	SAA, solar storms

5.1.2 Logic Flow

$$\boxed{\text{Measure } R_{\text{SEU}} \rightarrow \text{Select Level} \rightarrow \text{Apply ECC} \rightarrow \text{Monitor} \rightarrow \text{Adjust}} \quad (117)$$

5.2 Scrubbing Interval Calculation

5.2.1 Basic Formula

$$T_{\text{scrub}} < \frac{n_{\text{correctable}}}{R_{\text{SEU}} \cdot N_{\text{bits}}} \quad (118)$$

Table 10: Scrubbing Formula Field Definitions

Symbol	Name	Units	Meaning
T_{scrub}	Scrubbing period	seconds	Time between memory scans
$n_{\text{correctable}}$	Correction capability	errors	Max errors ECC can fix
R_{SEU}	SEU rate	errors/(bit·day)	Bit flip rate
N_{bits}	Protected memory	bits	Total bits under protection

5.2.2 Example

Given:

- RS-16 protection: $n_{\text{correctable}} = 8$ symbol errors
- SEU rate: $R_{\text{SEU}} = 5 \times 10^{-5}$ errors/(bit·day)
- Memory size: $N_{\text{bits}} = 10^6$ bits

$$T_{\text{scrub}} < \frac{8}{5 \times 10^{-5} \times 10^6} \text{ days} \quad (119)$$

$$< \frac{8}{50} \text{ days} \quad (120)$$

$$< 0.16 \text{ days} = 13,824 \text{ seconds} \quad (121)$$

With safety factor $\kappa = 0.3$:

$$T_{\text{scrub}} = 0.3 \times 13,824 = 4,147 \text{ s} \approx 69 \text{ minutes} \quad (122)$$

5.3 MBU-Corrected Scrubbing

5.3.1 Refined Formula

$$T_{\text{scrub}} < \frac{\kappa \cdot n_{\text{correctable}}}{R_{\text{SEU}} \cdot N_{\text{bits}} \cdot (1 + \beta_{\text{MBU}})} \quad (123)$$

Table 11: MBU Correction Parameters

Symbol	Name	Typical Value	Meaning
κ	Safety factor	0.3–0.5	Conservativeness
β_{MBU}	MBU clustering fraction	0.2–0.5	Extra upset fraction from MBU clustering

5.3.2 Physical Meaning

$$\beta_{\text{MBU}} = 0.3 \implies 30\% \text{ additional upset burden from clustered MBUs} \quad (124)$$

This effectively increases the rate:

$$R_{\text{effective}} = R_{\text{SEU}} \cdot (1 + 0.3) = 1.3 \cdot R_{\text{SEU}} \quad (125)$$

5.3.3 Example with MBU Correction

Same parameters as before, but $\beta_{\text{MBU}} = 0.3$:

$$T_{\text{scrub}} < \frac{0.3 \times 8}{5 \times 10^{-5} \times 10^6 \times (1 + 0.3)} \quad (126)$$

$$< \frac{2.4}{5 \times 10^{-5} \times 10^6 \times 1.3} \quad (127)$$

$$< \frac{2.4}{65} \text{ days} \quad (128)$$

$$< 0.0369 \text{ days} = 3,189 \text{ seconds} \approx 53 \text{ minutes} \quad (129)$$

5.4 Uncorrectable Error Probability

5.4.1 Poisson Model

Errors arrive as a Poisson process. The probability of observing exactly k errors in time t is:

$$P(k \text{ errors}) = \frac{(\lambda t)^k}{k!} e^{-\lambda t} \quad (130)$$

where $\lambda = R_{\text{SEU}} \cdot N_{\text{bits}}$ is the total error rate (errors per unit time).

The system becomes **uncorrectable** when more than $n_{\text{correctable}}$ errors accumulate:

$$P_{\text{uncorrectable}}(t) = P(k > n_{\text{correctable}}) = 1 - \sum_{k=0}^{n_{\text{correctable}}} \frac{(\lambda t)^k}{k!} e^{-\lambda t} \quad (131)$$

Logic:

- Errors arrive as Poisson process with rate λ

- ECC can correct up to $n_{\text{correctable}}$ errors
- System fails when $(n_{\text{correctable}} + 1)$ or more errors accumulate
- Formula is the complement of the Poisson CDF

Approximation for Large n : When $\lambda t \ll n_{\text{correctable}}$, the dominant term is:

$$P_{\text{uncorrectable}}(t) \approx \frac{(\lambda t)^{n_{\text{correctable}}+1}}{(n_{\text{correctable}} + 1)!} \quad (132)$$

5.4.2 Example

Given:

- $R_{\text{SEU}} = 5 \times 10^{-5}$ errors/(bit·day)
- $N_{\text{bits}} = 10^6$ bits
- $\lambda = 5 \times 10^{-5} \times 10^6 = 50$ errors/day
- $n_{\text{correctable}} = 8$ symbols (RS-16 can correct 8 symbol errors)
- Check probability at $t = 0.1$ days

Expected errors: $\lambda t = 50 \times 0.1 = 5$ errors

$$P_{\text{uncorrectable}}(0.1) = 1 - \sum_{k=0}^8 \frac{5^k}{k!} e^{-5} \quad (133)$$

$$= 1 - e^{-5} \left(1 + 5 + \frac{25}{2} + \frac{125}{6} + \dots + \frac{5^8}{8!} \right) \quad (134)$$

$$= 1 - e^{-5} \times 138.31 \quad (135)$$

$$= 1 - 0.00674 \times 138.31 \quad (136)$$

$$= 1 - 0.9319 \quad (137)$$

$$\approx 0.0681 \quad (138)$$

Interpretation: With 5 expected errors and ability to correct 8, there's approximately 6.8% chance of uncorrectable failure.

Conservative scrub interval to keep $P_{\text{uncorrectable}} < 0.01$:

For low failure probability, we want $\lambda t \ll n_{\text{correctable}}$. A practical rule:

$$T_{\text{scrub}} < \frac{n_{\text{correctable}}}{\lambda \cdot \text{safety_factor}} = \frac{8}{50 \times 3} = 0.053 \text{ days} = 4600 \text{ s} \approx 77 \text{ min} \quad (139)$$

With safety factor of 3, scrub every ~77 minutes to maintain high reliability.

6 Orbital Radiation Environment

6.1 Geomagnetic Cutoff Rigidity

6.1.1 Störmer Formula

Vertical cutoff rigidity:

$$R_c = \frac{M \cos^4 \lambda}{L^2} \quad (140)$$

Table 12: Geomagnetic Cutoff Parameters

Symbol	Name	Units	Value	Meaning
R_c	Cutoff rigidity	GV	0.1–15	Min momentum/charge
M	Dipole moment	$\text{GV} \cdot R_E^2$	59.6	Mag. field strength
λ	Geomag. latitude	degrees	−90 to 90	Magnetic coordinate
L	L-shell	R_E	1–10	McIlwain parameter
R_E	Earth radius	km	6371	Reference distance

6.1.2 Rigidity to Energy Conversion

For protons:

$$E = \sqrt{(R \cdot Z \cdot c)^2 + (m_p c^2)^2} - m_p c^2 \quad (141)$$

where:

- $Z = 1$ (proton charge)
- $m_p c^2 = 938.27 \text{ MeV}$ (proton rest energy)
- c = speed of light

6.1.3 Example: $R_c = 1 \text{ GV}$ for protons

$$E = \sqrt{(1000 \text{ MeV})^2 + (938.27 \text{ MeV})^2} - 938.27 \text{ MeV} \quad (142)$$

$$= \sqrt{1,000,000 + 880,350} - 938.27 \quad (143)$$

$$= 1371.18 - 938.27 \quad (144)$$

$$= 432.91 \text{ MeV} \quad (145)$$

6.1.4 Cutoff at Equator

$$R_c(\text{equator}) = \frac{M}{L^2} = \frac{59.6}{L^2} \text{ GV} \quad (146)$$

For ISS at $L \approx 1.06$:

$$R_c = \frac{59.6}{(1.06)^2} = 53.0 \text{ GV} \quad (147)$$

But effective cutoff is lower due to penumbra → **14.9 GV** (from document)

6.2 McIlwain L-Shell Parameter

6.2.1 Definition

$$L = \frac{r}{R_E \cos^2 \lambda_m} \quad (148)$$

where λ_m is magnetic latitude.

Field Lines: Particles spiral along magnetic field lines characterized by L :

$$r(\lambda_m) = L \cdot R_E \cdot \cos^2 \lambda_m \quad (149)$$

6.2.2 Example: ISS Orbit

Given:

- Altitude: 400 km
- Inclination: 51.6°
- At equatorial crossing: $\lambda_m = 0$

$$L = \frac{r}{R_E \cos^2(0)} = \frac{6371 + 400}{6371 \cdot 1} = \frac{6771}{6371} = 1.063 \quad (150)$$

6.2.3 Trapped Particle Flux

The AP-8 inspired model uses a Gaussian profile peaked at $L = 1.5$:

$$\Phi_{\text{trapped}}(L) \propto \exp\left(-\frac{(L - 1.5)^2}{0.5}\right) \quad (151)$$

Valid range: $1.15 \leq L \leq 6.6$ for protons, with an exponential energy rolloff $\propto e^{-E/E_0}$ ($E_0 = 40$ MeV).

6.3 South Atlantic Anomaly (SAA)

6.3.1 Geographic Location

Center: $(-29, -47) = 29^\circ\text{S}, 47^\circ\text{W}$ (off coast of Brazil)

Physical Cause: Offset between geographic and magnetic poles

6.3.2 Enhanced Flux

The implementation uses a smooth Gaussian enhancement profile (not discrete tiers):

$$\text{Enhancement}(\text{lat}, \text{lon}) = 1 + 99 \cdot \exp\left(-3 \left(\frac{\Delta\text{lat}^2}{\sigma_{\text{lat}}^2} + \frac{\Delta\text{lon}^2}{\sigma_{\text{lon}}^2} \right)\right) \quad (152)$$

where $\Delta\text{lat} = \text{lat} - (-29)$, $\Delta\text{lon} = \text{lon} - (-47)$, $\sigma_{\text{lat}} = 25$ (semi-minor), $\sigma_{\text{lon}} = 35$ (semi-major). The elliptic distance $r^2 = (\Delta\text{lat}/\sigma_{\text{lat}})^2 + (\Delta\text{lon}/\sigma_{\text{lon}})^2$ is checked: if $r^2 \geq 1$ the enhancement is $1 \times$ (outside SAA).

Table 13: SAA Enhancement Factor (approximate)

Location	Enhancement
Center ($r^2 = 0$)	$100\times$ (peak)
Mid-region ($r^2 \approx 0.3$)	$\sim 41\times$
Near boundary ($r^2 \approx 0.8$)	$\sim 10\times$
Outside SAA ($r^2 \geq 1$)	$1\times$

Table 14: ISS SAA Exposure

Parameter	Value	Calculation
Orbital period	92.7 min	From $T = 2\pi\sqrt{a^3/\mu}$
Orbits per day	15.5	$1440/92.7$
SAA crossings/day	~ 9	Document validation
Time in SAA	7.3%	$9 \times 12/1440$
Avg crossing duration	~ 12 min	Document value

6.3.3 ISS SAA Statistics

6.3.4 Ground Track Calculation

$$\text{Longitude} = \lambda_0 + \omega_E(t - t_0) - \Omega \quad (153)$$

$$\text{Latitude} = \arcsin(\sin i \cdot \sin(\omega + \nu)) \quad (154)$$

where:

- i = inclination (51.6° for ISS)
- ω_E = Earth rotation rate ($15^\circ/\text{hour}$)
- Ω = right ascension of ascending node
- ν = true anomaly

6.4 Galactic Cosmic Ray (GCR) Spectrum

6.4.1 Adams LET-Based Model (Implemented)

The production `GCRSpectrum` class uses an Adams-style LET-based power-law model rather than a per-species energy spectrum:

$$\frac{d\Phi}{dLET} = A \cdot \gamma \cdot \text{LET}^{-(\gamma+1)} \quad (155)$$

where $\gamma = 1.9$ and A is linearly interpolated with solar modulation:

$$A = A_{\max} + (A_{\min} - A_{\max}) \cdot \frac{1200 - \Phi_{\text{mod}}}{800} \quad (156)$$

Note: This is a simplified aggregate spectrum that folds all heavy-ion species into a single LET distribution, rather than summing individual elemental spectra as in the full CREME96 model. The integration in `gcr_heavy_ion_rate()` computes $\int \Phi(\text{LET}) \cdot \sigma(\text{LET}) d\text{LET}$ directly.

6.4.2 Solar Modulation

Solar modulation affects flux amplitude via linear interpolation between solar minimum ($\Phi_{\text{mod}} \approx 400$ MV, higher flux) and solar maximum ($\Phi_{\text{mod}} \approx 1200$ MV, lower flux):

Table 15: GCR Model Parameters (as implemented)

Symbol	Name	Value
Φ	Differential LET flux	particles/(cm ² ·s·sr·(MeV·cm ² /mg))
LET	Linear energy transfer	MeV·cm ² /mg
γ	Power law index	1.9
$A_{\min}$	Solar minimum coeff	0.24
$A_{\max}$	Solar maximum coeff	0.08
Φ_{mod}	Solar modulation	400–1200 MV

Table 16: Solar Modulation by Cycle Phase

Solar Cycle Phase	Φ_{mod} (MV)	GCR Flux Level
Solar minimum	400–550	Maximum (worst case)
Solar moderate	600–800	Nominal
Solar maximum	1000–1200	Minimum (best case)

7 Integrated SEU Rate Calculation

This section describes the **production SEU rate calculation** using validated engineering models (Weibull, Bendel, NASA flux). For quantum-enhanced calculations, see Section 2 and use:

```
QuantumEnhancedRadiation::calculateQuantumChargeDeposition()
```

7.1 Complete Calculation

7.1.1 Full Formula

$$\begin{aligned}
 R_{\text{SEU}} = & \underbrace{\int_0^\infty \Phi_{\text{GCR}}(E) \cdot \sigma_{\text{heavy}}(L(E)) dE}_{\text{GCR heavy ions}} \\
 & + \underbrace{\int_0^\infty \Phi_{\text{proton}}(E) \cdot \sigma_p(E) dE}_{\text{Trapped + GCR protons}}
 \end{aligned} \tag{157}$$

7.1.2 Step-by-Step for LEO (ISS)

Step 1: GCR Heavy Ion Contribution

The implementation integrates directly over LET using the Adams spectrum (not per-element):

$$R_{\text{GCR,heavy}} = 4\pi \cdot 86400 \int_{\text{LET}_{\min}}^{\text{LET}_{\max}} \frac{d\Phi}{d\text{LET}}(\text{LET}) \cdot \sigma_{\text{Weibull}}(\text{LET}) d\text{LET} \tag{158}$$

where $\text{LET}_{\min} = 0.1$ and $\text{LET}_{\max} = 100$ MeV·cm²/mg.

Numerical integration (midpoint rule with 100 LET bins):

$$\int f(\text{LET}) d\text{LET} \approx \sum_{i=0}^{99} f(\text{LET}_i) \cdot \Delta\text{LET}, \quad \text{LET}_i = \text{LET}_{\min} + (i + 0.5) \cdot \Delta\text{LET} \tag{159}$$

Geomagnetic shielding is applied as a LET cutoff derived from the minimum proton energy at the spacecraft's latitude.

Step 2: Proton Contribution

$$R_{\text{proton}} = \int_A^{E_{\text{max}}} \Phi_{\text{proton,total}}(E) \cdot \sigma_p(E) dE \quad (160)$$

$$(161)$$

$$\Phi_{\text{proton,total}} = \Phi_{\text{GCR,proton}} + \Phi_{\text{trapped,proton}} \quad (162)$$

For ISS outside SAA: $\Phi_{\text{trapped}} \approx 0$ (too low altitude)

Typical result: $R_{\text{proton}} \approx 1.1 \times 10^{-5}$ errors/(bit·day)

Step 3: Total SEU Rate

$$R_{\text{SEU,total}} = R_{\text{GCR,heavy}} + R_{\text{proton}} = 4.2 \times 10^{-5} + 1.1 \times 10^{-5} = 5.3 \times 10^{-5} \text{ errors/}(\text{bit} \cdot \text{day}) \quad (163)$$

Step 4: SAA Enhancement

Inside SAA, the enhancement follows the Gaussian model from Section ?? (up to $100\times$ at center). The code applies `SouthAtlanticAnomaly::enhancement_factor(lat, lon)` as a multiplier on trapped fluxes.

Step 5: Orbit-Averaged Rate

The implementation in `get_orbit_average_seu_rate()` samples latitude/longitude points around the orbit and applies the spatial SAA enhancement factor at each point, rather than using a simple two-level weighted average. Conceptually:

$$R_{\text{avg}} = \frac{1}{N} \sum_{i=1}^N R_{\text{SEU}}(\text{lat}_i, \text{lon}_i) \quad (164)$$

where each R_{SEU} includes the position-dependent SAA enhancement.

7.2 Mission-Specific SEU Rates

Table 17: SEU Rates by Mission

Mission	Alt (km)	Inc	Avg SEU Rate	Dominant
ISS	400	51.6°	5.3×10^{-5}	GCR heavy (80%)
LEO Polar	600	90°	5.3×10^{-5}	GCR heavy (80%)
GPS (MEO)	20,200	55°	4.5×10^{-5}	GCR heavy (90%)
GEO	35,786	0°	4.5×10^{-5}	GCR heavy (95%)
ISS in SAA	400	51.6°	5.4×10^{-3}	Trapped p (99%)

Key Insight: GCR dominates everywhere except SAA

8 Memory Overhead and Performance

8.1 Memory Layout

8.1.1 Single Float with RS-16

Memory Layout:

[Original float: 4 bytes] [RS-16 ECC: 16 bytes] [Padding: 4-12 bytes]
= 24-32 bytes total (600-700% overhead)

8.1.2 Cache Line Impact

Modern CPUs use 64-byte cache lines.

Floats per cache line:

$$\text{Floats per cache line} = \left\lfloor \frac{64 \text{ bytes}}{24 \text{ bytes/protected float}} \right\rfloor = 2 \text{ floats} \quad (165)$$

Without protection:

$$\text{Floats per cache line} = \frac{64}{4} = 16 \text{ floats} \quad (166)$$

Cache efficiency degradation:

$$\frac{2}{16} = 12.5\% \quad (87.5\% \text{ loss}) \quad (167)$$

8.2 Computational Overhead

Table 18: Encoding/Decoding Times

Operation	Hamming	RS-16	Ratio
Encode (per float)	65 ns	1,300 ns	20×
Decode (per float)	70 ns	1,400 ns	20×

8.2.1 For 1M Weight Neural Network

$$T_{\text{encode,Hamming}} = 10^6 \times 65 \text{ ns} = 65 \text{ ms} \quad (168)$$

$$T_{\text{encode,RS-16}} = 10^6 \times 1300 \text{ ns} = 1.3 \text{ s} \quad (169)$$

8.2.2 Scrubbing Overhead

If scrubbing every 1 hour:

$$\text{Duty cycle} = \frac{1.3 \text{ s}}{3600 \text{ s}} = 0.036\% \quad (\text{negligible}) \quad (170)$$

8.3 Block Encoding for Large Data

8.3.1 RS(255, k) Constraint

$$k_{\text{max}} = 255 - 2t = 255 - 16 = 239 \text{ data symbols} \quad (171)$$

For 8-bit symbols: 239 bytes max data per codeword

8.3.2 Large Layer Encoding

For 1024 floats = 4096 bytes:

$$\text{Blocks needed} = \left\lceil \frac{4096}{239} \right\rceil = 18 \text{ blocks} \quad (172)$$

$$\text{Total memory} = 18 \times 255 = 4590 \text{ bytes} \quad (173)$$

$$\text{Overhead} = \frac{4590 - 4096}{4096} = 12\% \quad (174)$$

8.3.3 Block Interleaving Strategy

Instead of:

[Block 1: Data | ECC] [Block 2: Data | ECC] [Block 3: Data | ECC] ...

Use interleaved layout:

[B1_D0, B2_D0, B3_D0, ..., B1_D1, B2_D1, ...]

Benefit: Burst errors spread across multiple codewords

$$4\text{-byte burst} \rightarrow 1 \text{ error per block (4 blocks)} \rightarrow \text{All correctable} \quad (175)$$

9 Implementation Details

9.1 Type Safety with Templates

9.1.1 C++ Template Example

```
template <typename T, uint8_t SymbolSize = 8, uint8_t ECCSymbols = 8>
class AdvancedReedSolomon {
    static_assert(std::is_trivially_copyable_v<T>,
                  "T must be trivially copyable for memcpy safety");
    static_assert(SymbolSize > 0 && SymbolSize <= 16,
                  "Symbol size must be 1-16 bits");

    constexpr size_t correction_capability() const {
        return ECCSymbols / 2;
    }
};
```

Note: The `sizeof(T) <= 247` constraint is enforced by `is_rs_protectable<T>` in `multi_bit_protection.hpp`, not in the codec class itself.

9.1.2 Compile-Time Checks

$$\text{sizeof}(T) \leq 255 - \text{ECCSymbols} \quad (176)$$

For RS-8: $\text{sizeof}(T) \leq 255 - 8 = 247$ bytes

9.1.3 Type Aliases

Protection tiers are managed through `RSProtectedValue` in `multi_bit_protection.hpp`, which wraps the codec:

```
// In multi_bit_protection.hpp:
enum class RSCorrectionTier { LIGHT = 4, STANDARD = 6, HEAVY = 8 };

template<typename T, RSCorrectionTier Tier>
class RSProtectedValue; // wraps AdvancedReedSolomon<T, 8, ecc_symbols>

// Direct codec aliases (in advanced_reed_solomon.hpp):
template<typename T> using RS8Bit8Sym = AdvancedReedSolomon<T, 8, 8>; // t=4
template<typename T> using RS8Bit16Sym = AdvancedReedSolomon<T, 8, 16>; // t=8
```

9.2 Zero-Cost Abstraction

9.2.1 RAII Pattern

```
class RSProtectedValue {
    std::vector<uint8_t> encoded_;

public:
    RSProtectedValue(float value) {
        encoded_ = rs_.encode(value); // Encode on construction
    }

    ~RSProtectedValue() {
        // Automatic cleanup
    }
}
```

```

    operator float() const {
        auto result = rs_.decode(encoded_);
        return result.value_or(0.0f); // Decode on access
    }
};

```

9.2.2 Zero Runtime Cost

Compiler optimizations (inlining, constant propagation) eliminate abstraction overhead:

$$\text{Performance}(\text{abstracted code}) = \text{Performance}(\text{hand-written code}) \quad (177)$$

9.3 Syndrome Verification Logic

9.3.1 Implementation

```

bool verify_correction(const std::vector<uint8_t>& corrected) {
    auto syndromes = calculate_syndromes(corrected);
    return std::all_of(syndromes.begin(), syndromes.end(),
        [](uint8_t s) { return s == 0; });
}

```

9.3.2 Mathematical Guarantee

$$\text{All } S_i = 0 \iff \text{No errors remain} \iff \text{Correct decoding} \quad (178)$$

This catches:

- Wrong error positions
- Incorrect error magnitudes
- Algebraic errors in solver

10 Validation Methodology

10.1 Monte Carlo Simulation

10.1.1 Test Structure

For each of 10,000 trials:

1. Generate random float: $v \sim U(0, 1)$
2. Encode: $c = \text{Encode}(v)$
3. Inject errors: $r = c \oplus e$ where e has $\leq t$ non-zero symbols
4. Decode: $\hat{v} = \text{Decode}(r)$
5. Verify: $|\hat{v} - v| < \epsilon$

10.1.2 Success Criterion

$$P_{\text{success}} = \frac{\sum \mathbb{I}[|\hat{v}_i - v_i| < \epsilon]}{N_{\text{trials}}} \geq 0.9999 \quad (179)$$

Document Results: 100% success across 160,000 trials

$$\text{Confidence interval (95\%)} : [0.9999, 1.0000] \quad (180)$$

10.2 Error Injection Patterns

10.2.1 Random Errors

$$P(\text{error at position } i) = \frac{t}{n} \quad (\text{uniform}) \quad (181)$$

10.2.2 Burst Errors

Consecutive positions with exponential decay:

$$P(\text{burst length} = k) = \lambda e^{-\lambda k} \quad (182)$$

where $\lambda \approx 0.5$.

10.2.3 MBU Realistic Patterns (Future Work)

- **Word-line MBU:** Same byte, adjacent bits
- **Column MBU:** Same bit position, adjacent bytes
- **Diagonal MBU:** Heavy ion track (45° through die)

11 Summary Tables

11.1 Complete Symbol Reference

Table 19: Complete Symbol Reference

Symbol	Name	Units	Domain
R_{SEU}	SEU rate	errors/(bit·day)	10^{-7} to 10^{-2}
$\sigma(E)$	Cross-section	cm^2/bit	10^{-14} to 10^{-8}
$\Phi(E)$	Particle flux	particles/($\text{cm}^2 \cdot \text{s} \cdot \text{MeV}$)	10^{-4} to 10^4
L	L-shell parameter	Earth radii	1 to 10
R_c	Geomagnetic cutoff	GV	0.1 to 15
t	Correction capability	symbols	2 to 8
n	Codeword length	symbols	7 to 255
α	Primitive element	GF(256)	2 (typically)
$\Lambda(x)$	Error locator polynomial	GF(256)[x]	degree $\leq t$
T_{scrub}	Scrubbing interval	seconds	60 to 100,000
β_{MBU}	MBU clustering fraction	dimensionless	0.2 to 0.5
κ	Safety factor	dimensionless	0.3 to 0.5
m	Symbol size	bits	8 (typical)
k	Message length	symbols	Variable
S_i	Syndrome	GF(256)	0 to 255
e_j	Error magnitude	GF(256)	0 to 255
X_j	Error locator	GF(256)	α^j

11.2 Algorithm Complexity Summary

Table 20: Decoding Algorithm Complexities

Algorithm	Time	Space	When to Use
Peterson	$O(n)$	$O(1)$	Single error only
Brute-Force 2	$O(n^2)$	$O(1)$	2 errors
Brute-Force 3	$O(n^3)$	$O(1)$	3 errors
Berlekamp-Massey	$O(t^2)$	$O(t)$	General, $\leq t$ errors
Chien Search	$O(n \cdot t)$	$O(t)$	Find error positions
Forney	$O(t^2)$	$O(t)$	Find error magnitudes

11.3 Protection Configuration Summary

Table 21: RadML Protection Configurations

Level	Scheme	t	Overhead	T_{scrub}	Use Case
MINIMAL	Parity	0	12.5%	N/A	Detection
MODERATE	Hamming	1	75%	~1 day	LEO nominal
HIGH	RS-8	4	200%	~18 hr	GEO
VERY_HIGH	RS-16	8	400%	~8 hr	SAA/SPE

12 Code Implementation Cross-Reference

This section demonstrates how the mathematical formulations in RadML translate directly to C++ implementations. Each subsection shows the mathematical formula alongside its corresponding code implementation with detailed explanations.

12.1 Galois Field Arithmetic Implementation

12.1.1 Field Definition and Primitive Polynomial

Mathematical Definition:

$$\text{GF}(2^8) \text{ with } p(x) = x^8 + x^4 + x^3 + x^2 + 1 \quad (0x11D) \quad (183)$$

C++ Implementation (include/rad_ml/neural/galois_field.hpp):

```
// Line 37-44: Template class definition
template <uint8_t m, uint16_t Poly>
class GaloisField {
    static constexpr uint32_t field_size = (1ULL << m); // 2^m = 256
    static constexpr element_t field_mask = field_size - 1; // 0xFF
    static constexpr uint16_t primitive_poly = Poly; // 0x11D
};

// Line 743: Standard GF(256) instantiation
using GF256 = GaloisField<8, 0x11D>;
```

Translation Notes:

- Template parameter m=8 creates $2^8 = 256$ elements
- 0x11d encodes $x^8 + x^4 + x^3 + x^2 + 1$ (bits 8,4,3,2,0 set)
- field_mask = 0xFF used for modular arithmetic

12.1.2 Addition in GF(256)

Mathematical Definition:

$$a \oplus b = a \text{ XOR } b \quad (184)$$

C++ Implementation:

```
// Line 62: Addition is simply XOR
constexpr element_t add(element_t a, element_t b) const {
    return a ^ b;
}

// Line 71: Subtraction is identical (self-inverse property)
constexpr element_t subtract(element_t a, element_t b) const {
    return a ^ b;
}
```

Why XOR? In characteristic-2 fields, $1 + 1 = 0$, so addition follows XOR truth table. This is computationally efficient (single CPU instruction).

12.1.3 Multiplication Using Log Tables

Mathematical Definition:

$$a \cdot b = \alpha^{(\log_\alpha(a) + \log_\alpha(b)) \bmod 255} \quad (185)$$

C++ Implementation:

```
// Lines 80-87: Multiplication via log-antilog tables
element_t multiply(element_t a, element_t b) const {
    // Handle special cases: anything times 0 is 0
    if (a == 0 || b == 0) return 0;

    // Use log-antilog method: a*b = exp(log(a) + log(b))
    return exp_table[(log_table[a] + log_table[b]) % (field_size - 1)];
}
```

Performance: Log tables convert multiplication to addition + table lookup, reducing $O(n^2)$ polynomial multiplication to $O(1)$.

12.1.4 Table Initialization

Mathematical Basis: The primitive element $\alpha = 2$ generates all non-zero elements:

$$\{\alpha^0, \alpha^1, \alpha^2, \dots, \alpha^{254}\} = \text{GF}(256) \setminus \{0\} \quad (186)$$

C++ Implementation:

```
// Lines 678-707: Initialize exp and log lookup tables
void initialize_tables() {
    element_t x = 1; // Start with alpha^0 = 1

    for (uint32_t i = 0; i < field_size - 1; ++i) {
        exp_table[i] = x; // exp_table[i] = alpha^i

        // Multiply by alpha (which is 2 in this field)
        x = multiply_no_lut(x, 2);

        // Reduce modulo primitive polynomial if overflow
        if (x >= field_size) {
            x ^= (primitive_poly & field_mask);
        }
    }

    // Build log table (inverse of exp table)
    for (uint32_t i = 0; i < field_size - 1; ++i) {
        log_table[exp_table[i]] = static_cast<element_t>(i);
    }
}
```

12.2 Hamming(7,4) Code Implementation

12.2.1 Encoding

Mathematical Definition:

$$p_1 = d_1 \oplus d_2 \oplus d_4 \quad (187)$$

$$p_2 = d_1 \oplus d_3 \oplus d_4 \quad (188)$$

$$p_3 = d_2 \oplus d_3 \oplus d_4 \quad (189)$$

C++ Implementation (include/rad_ml/neural/adaptive_protection.hpp):

```
// Lines 674-687: Hamming(7,4) encoder
static uint8_t hamming_encode_nibble(uint8_t nibble) {
    // Extract 4 data bits
    uint8_t d1 = (nibble >> 0) & 1;
    uint8_t d2 = (nibble >> 1) & 1;
    uint8_t d3 = (nibble >> 2) & 1;
    uint8_t d4 = (nibble >> 3) & 1;

    // Calculate parity bits (XOR = addition in GF(2))
    uint8_t p1 = d1 ^ d2 ^ d4; // Covers positions 1,3,5,7
    uint8_t p2 = d1 ^ d3 ^ d4; // Covers positions 2,3,6,7
    uint8_t p3 = d2 ^ d3 ^ d4; // Covers positions 4,5,6,7

    // Construct codeword: [p1, p2, d1, p3, d2, d3, d4]
    return (p1 << 0) | (p2 << 1) | (d1 << 2) |
        (p3 << 3) | (d2 << 4) | (d3 << 5) | (d4 << 6);
}
```

Bit Position Mapping:

Bit 0	Bit 1	Bit 2	Bit 3	Bit 4	Bit 5	Bit 6
p_1	p_2	d_1	p_3	d_2	d_3	d_4

12.2.2 Syndrome Calculation and Error Correction

Mathematical Definition:

$$s_1 = r_1 \oplus r_3 \oplus r_5 \oplus r_7 \quad (190)$$

$$s_2 = r_2 \oplus r_3 \oplus r_6 \oplus r_7 \quad (191)$$

$$s_3 = r_4 \oplus r_5 \oplus r_6 \oplus r_7 \quad (192)$$

C++ Implementation:

```
// Lines 695-721: Hamming(7,4) decoder with error correction
static std::tuple<uint8_t, bool> hamming_decode_nibble(uint8_t codeword) {
    // Extract all 7 bits
    uint8_t p1 = (codeword >> 0) & 1;
    uint8_t p2 = (codeword >> 1) & 1;
    uint8_t d1 = (codeword >> 2) & 1;
    uint8_t p3 = (codeword >> 3) & 1;
    uint8_t d2 = (codeword >> 4) & 1;
    uint8_t d3 = (codeword >> 5) & 1;
    uint8_t d4 = (codeword >> 6) & 1;

    // Calculate syndrome bits
    uint8_t s1 = p1 ^ d1 ^ d2 ^ d4; // Check parity group 1
    uint8_t s2 = p2 ^ d1 ^ d3 ^ d4; // Check parity group 2
    uint8_t s3 = p3 ^ d2 ^ d3 ^ d4; // Check parity group 3

    // Syndrome gives error position (binary encoding)
    uint8_t error_pos = (s3 << 2) | (s2 << 1) | s1;

    // Correct single-bit error if detected
    if (error_pos != 0 && error_pos <= 7) {
        codeword ^= (1 << (error_pos - 1)); // Flip the erroneous bit
        // Re-extract data bits after correction
    }
}
```

```

    d1 = (codeword >> 2) & 1;
    d2 = (codeword >> 4) & 1;
    d3 = (codeword >> 5) & 1;
    d4 = (codeword >> 6) & 1;
}

// Return decoded nibble and correction flag
return {(d1 << 0) | (d2 << 1) | (d3 << 2) | (d4 << 3),
        error_pos != 0};
}

```

Key Insight: The syndrome value directly encodes the error position in binary. For example, syndrome $[1, 0, 1]_2 = 5$ means error at position 5.

12.3 Weibull Cross-Section Model Implementation

Mathematical Definition:

$$\sigma(L) = \begin{cases} \sigma_{\text{sat}} \cdot \left[1 - \exp\left(-\left(\frac{L-L_{\text{th}}}{W}\right)^s\right) \right] & \text{if } L > L_{\text{th}} \\ 0 & \text{if } L \leq L_{\text{th}} \end{cases} \quad (193)$$

C++ Implementation (include/rad_ml/physics/radiation_physics.hpp):

```

// Lines 83-102: Weibull parameter structure
struct WeibullParameters {
    double sigma_sat;    // Saturation cross-section (cm^2/bit)
    double let_threshold; // LET threshold (MeV*cm^2/mg)
    double width;        // Weibull width W
    double shape;        // Weibull shape s

    // Factory methods for common device types
    static WeibullParameters cmos_65nm() {
        return {1e-8, 0.5, 15.0, 2.0};
    }
    static WeibullParameters cmos_28nm() {
        return {5e-8, 0.2, 10.0, 1.5};
    }
};

// Lines 136-144: Weibull cross-section calculation
inline double weibull_cross_section(double let,
                                    const WeibullParameters& params) {
    // Below threshold: no upsets possible
    if (let <= params.let_threshold) {
        return 0.0;
    }

    // Normalized distance above threshold
    double x = (let - params.let_threshold) / params.width;

    // Weibull CDF formula
    return params.sigma_sat * (1.0 - std::exp(-std::pow(x, params.shape)));
}

```

Physical Interpretation:

1. `let <= let_threshold`: Particle doesn't deposit enough charge
2. `x = (let - let_threshold) / width`: Normalized "excess" energy

3. `pow(x, shape)`: Controls transition steepness
4. `1 - exp(...)`: Saturates to σ_{sat} at high LET

12.4 Bendel Proton Model Implementation

Mathematical Definition:

$$\sigma_p(E) = \begin{cases} \sigma_{\infty} \cdot \left[1 - \exp \left(-0.18 \left(\sqrt{E} - \sqrt{A} \right) \right) \right]^4 & \text{if } E > A \\ 0 & \text{if } E \leq A \end{cases} \quad (194)$$

C++ Implementation:

```
// Lines 156-176: Bendel proton cross-section
inline double bendel_proton_cross_section(double energy_mev,
                                         const BendelParameters& params) {
    // Below threshold: nuclear reactions don't occur
    if (energy_mev <= params.A) {
        return 0.0;
    }

    // Bendel two-parameter model
    double sqrt_E = std::sqrt(energy_mev);
    double sqrt_A = std::sqrt(params.A);

    // Y factor from Bendel formula
    double Y = 0.18 * (sqrt_E - sqrt_A);
    if (Y <= 0) return 0.0;

    // Fourth power gives characteristic S-curve
    double term = 1.0 - std::exp(-Y);
    double sigma = params.sigma_inf * std::pow(term, 4);

    return sigma; // cm^2/bit
}
```

Why Square Root? Quantum mechanics: nuclear reaction cross-sections scale with de Broglie wavelength $\lambda \propto 1/\sqrt{E}$.

12.5 Reed-Solomon Encoding Implementation

12.5.1 Generator Polynomial Construction

Mathematical Definition:

$$g(x) = \prod_{i=0}^{2t-1} (x - \alpha^i) \quad (195)$$

C++ Implementation (include/rad_ml/neural/galois_field.hpp):

```
// Lines 163-185: Generate RS generator polynomial
std::vector<element_t> rs_generator_poly(uint8_t nsym) const {
    // Start with g(x) = 1
    std::vector<element_t> g = {1};

    // Multiply by (x - alpha^i) for i = 0 to nsym-1
    for (uint8_t i = 0; i < nsym; ++i) {
        std::vector<element_t> g_new(g.size() + 1, 0);
```

```

// Multiply g(x) by x (shift coefficients)
std::copy(g.begin(), g.end(), g_new.begin());

// Add g(x) * alpha^i (subtract in GF = add = XOR)
for (size_t j = 0; j < g.size(); ++j) {
    g_new[j + 1] = add(g_new[j + 1],
                      multiply(g[j], exp_table[i]));
}

g = std::move(g_new);
}

return g; // Coefficients [g_0, g_1, ..., g_nsym]
}

```

Example: For RS(n,k) with $t = 2$ (nsym=4):

$$g(x) = (x - \alpha^0)(x - \alpha^1)(x - \alpha^2)(x - \alpha^3) \quad (196)$$

$$= (x - 1)(x - 2)(x - 4)(x - 8) \quad (197)$$

$$= x^4 + 15x^3 + 54x^2 + 120x + 64 \quad (\text{in GF}(256)) \quad (198)$$

12.5.2 Systematic Encoding

Mathematical Definition:

1. Shift message: $x^{n-k} \cdot m(x)$
2. Compute remainder: $r(x) = [x^{n-k} \cdot m(x)] \bmod g(x)$
3. Codeword: $c(x) = x^{n-k} \cdot m(x) - r(x)$

C++ Implementation (include/rad_ml/neural/advanced_reed_solomon.hpp):

```

// Lines 402-439: Compute ECC symbols via polynomial division
std::vector<element_t> compute_ecc_symbols(
    const std::vector<element_t>& message) const {

    // Remainder register (initially zero)
    std::vector<element_t> remainder(ECCSymbols, 0);

    // Process each message symbol (systematic encoding)
    for (size_t i = 0; i < data_symbols; ++i) {
        // Feedback = message[i] XOR remainder[0]
        element_t feedback = field_.add(message[i], remainder[0]);

        if (feedback != 0) {
            // Shift and apply generator polynomial
            for (size_t j = 0; j < ECCSymbols - 1; ++j) {
                remainder[j] = field_.add(
                    remainder[j + 1],
                    field_.multiply(feedback, generator_poly_[j + 1])
                );
            }
            remainder[ECCSymbols - 1] = field_.multiply(
                feedback, generator_poly_[ECCSymbols]
            );
        } else {
            // Just shift remainder
            for (size_t j = 0; j < ECCSymbols - 1; ++j) {

```

```

        remainder[j] = remainder[j + 1];
    }
    remainder[ECCSymbols - 1] = 0;
}

return remainder; // These are the ECC symbols
}

```

Implementation Note: This is a Linear Feedback Shift Register (LFSR) implementation of polynomial division, which is hardware-efficient.

12.6 Berlekamp-Massey Algorithm Implementation

Mathematical Basis: Find shortest $\Lambda(x)$ satisfying:

$$S_{i+v} = \sum_{k=1}^v \Lambda_k S_{i+v-k} \quad (\text{in GF}(2^m), \text{ where } -x = x) \quad (199)$$

C++ Implementation (include/rad_ml/neural/galois_field.hpp):

```

// Lines 243-317: Berlekamp-Massey algorithm
std::tuple<std::vector<element_t>, std::vector<element_t>>
rs_find_error_locator(const std::vector<element_t>& syndromes,
                      uint8_t nsym) const {

    std::vector<element_t> C = {1}; // Error locator Lambda(x)
    std::vector<element_t> B = {1}; // Previous locator (backup)
    size_t L = 0; // Current error count estimate
    element_t b_prev = 1; // Previous discrepancy
    size_t shift = 1; // Steps since L changed

    for (uint8_t n = 0; n < nsym; ++n) {
        // Compute discrepancy: d = S[n+1] + sum(C[i] * S[n+1-i])
        element_t d = syndromes[n + 1];
        for (size_t i = 1; i < C.size() && i <= L; ++i) {
            if (n + 1 >= i) {
                d = add(d, multiply(C[i], syndromes[n + 1 - i]));
            }
        }

        if (d == 0) {
            shift++; // No update needed
        } else {
            // T(x) = C(x) - (d/b) * x^shift * B(x)
            std::vector<element_t> T = C;
            element_t coeff = divide(d, b_prev);

            if (T.size() < B.size() + shift) {
                T.resize(B.size() + shift, 0);
            }

            for (size_t i = 0; i < B.size(); ++i) {
                T[i + shift] = add(T[i + shift],
                                   multiply(coeff, B[i]));
            }

            if (2 * L <= n) {

```



```

        L = n + 1 - L; // Increase error estimate
        B = C;
        b_prev = d;
        shift = 1;
    } else {
        shift++;
    }

    C = T;
}

// Also compute error evaluator Omega(x)
// ... (omega computation follows)

return {C, omega};
}

```

12.7 Brute-Force 2-Error Decoder Implementation

Mathematical Definition: Solve the 2×2 system:

$$\begin{bmatrix} X_1 & X_2 \\ X_1^2 & X_2^2 \end{bmatrix} \begin{bmatrix} e_1 \\ e_2 \end{bmatrix} = \begin{bmatrix} S_1 \\ S_2 \end{bmatrix} \quad (200)$$

Cramer's Rule Solution:

$$\det = X_1 X_2 (X_2 \oplus X_1) \quad (201)$$

$$e_1 = \frac{S_1 X_2^2 \oplus S_2 X_2}{\det} \quad (202)$$

$$e_2 = \frac{S_1 X_1^2 \oplus S_2 X_1}{\det} \quad (203)$$

C++ Implementation:

```

// Lines 511-564: Brute-force 2-error decoder
// Try all pairs of positions (O(n^2) complexity)
for (size_t p1 = 0; p1 < n; ++p1) {
    for (size_t p2 = p1 + 1; p2 < n; ++p2) {
        // Position exponents
        size_t k1 = n - 1 - p1;
        size_t k2 = n - 1 - p2;

        // Error locators: X = alpha^k
        element_t X1 = exp_table[k1 % (field_size - 1)];
        element_t X2 = exp_table[k2 % (field_size - 1)];
        element_t X1_2 = multiply(X1, X1);
        element_t X2_2 = multiply(X2, X2);

        // Determinant: det = X1*X2*(X1 XOR X2)
        element_t det = multiply(multiply(X1, X2), add(X2, X1));
        if (det == 0) continue; // Singular matrix

        // Cramer's rule (note: subtraction = addition in GF(2^m))
        element_t e1 = divide(
            add(multiply(syndromes[1], X2_2),
                multiply(syndromes[2], X2)),
            det

```

```

    );
    element_t e2 = divide(
        add(multiply(syndromes[2], X1),
            multiply(syndromes[1], X1_2)),
        det
    );

    // Verify solution by checking all syndromes
    std::vector<element_t> candidate = msg;
    candidate[p1] = add(candidate[p1], e1);
    candidate[p2] = add(candidate[p2], e2);

    auto new_syndromes = rs_calc_syndromes(candidate, nsym);

    bool all_zero = true;
    for (uint8_t i = 0; i < nsym && all_zero; ++i) {
        if (new_syndromes[i] != 0) all_zero = false;
    }

    if (all_zero) return candidate; // Valid correction!
}
}

```

12.8 Adaptive Protection Level Selection

Mathematical Definition:

$$\text{ProtectionLevel}(R_{\text{SEU}}) = \begin{cases} \text{VERY_HIGH} & \text{if } R_{\text{SEU}} > 10^{-3} \\ \text{HIGH} & \text{if } 10^{-4} < R_{\text{SEU}} \leq 10^{-3} \\ \text{MODERATE} & \text{if } 10^{-5} < R_{\text{SEU}} \leq 10^{-4} \\ \text{MINIMAL} & \text{if } R_{\text{SEU}} \leq 10^{-5} \end{cases} \quad (204)$$

C++ Implementation (include/rad_ml/neural/adaptive_protection.hpp):

```

// Lines 443-461: Adaptive protection level selection
void adapt_to_environment() {
    double current_seu_rate = get_current_seu_probability();

    if (current_seu_rate > 1e-3) {
        // SAA/SPE: Maximum protection
        protection_level_ = AdaptiveProtectionLevel::VERY_HIGH;
    }
    else if (current_seu_rate > 1e-4) {
        // GEO or storm conditions
        protection_level_ = AdaptiveProtectionLevel::HIGH;
    }
    else if (current_seu_rate > 1e-5) {
        // Normal LEO operations
        protection_level_ = AdaptiveProtectionLevel::MODERATE;
    }
    else {
        // Deep space, low flux
        protection_level_ = AdaptiveProtectionLevel::MINIMAL;
    }
}

```

Design Rationale: Each order of magnitude increase in SEU rate triggers the next protection level, balancing overhead against reliability.

12.9 South Atlantic Anomaly Model

Mathematical Definition: Gaussian enhancement centered at SAA:

$$\text{Enhancement} = 1 + 99 \cdot \exp\left(-3 \cdot \left(\frac{\Delta\text{lat}^2}{\sigma_{\text{lat}}^2} + \frac{\Delta\text{lon}^2}{\sigma_{\text{lon}}^2}\right)\right) \quad (205)$$

C++ Implementation (include/rad_ml/physics/radiation_physics.hpp):

```
// Lines 606-653: South Atlantic Anomaly model
class SouthAtlanticAnomaly {
public:
    // SAA center (2024, drifting ~0.3-0.5 deg/year westward)
    static constexpr double CENTER_LAT = -29.0; // degrees
    static constexpr double CENTER_LON = -47.0; // degrees
    static constexpr double SEMI_MAJOR = 35.0; // longitude extent
    static constexpr double SEMI_MINOR = 25.0; // latitude extent

    static double enhancement_factor(double lat, double lon) {
        // Normalized distance from SAA center
        double dlat = (lat - CENTER_LAT) / SEMI_MINOR;
        double dlon = normalize_longitude_diff(lon - CENTER_LON)
            / SEMI_MAJOR;

        double r2 = dlat * dlat + dlon * dlon;

        if (r2 >= 1.0) return 1.0; // Outside SAA

        // Gaussian enhancement profile: peak ~100x at center
        return 1.0 + 99.0 * std::exp(-r2 * 3.0);
    }

    static bool inside(double lat, double lon) {
        double dlat = (lat - CENTER_LAT) / SEMI_MINOR;
        double dlon = normalize_longitude_diff(lon - CENTER_LON)
            / SEMI_MAJOR;
        return (dlat * dlat + dlon * dlon) < 1.0;
    }
};
```

Physical Note: The SAA is caused by the 500 km offset between Earth’s geographic and magnetic centers, causing the inner Van Allen belt to dip to LEO altitudes over South America.

12.10 Scrubbing Interval Calculation

Mathematical Definition:

$$T_{\text{scrub}} < \frac{\kappa \cdot n_{\text{correctable}}}{R_{\text{SEU}} \cdot N_{\text{bits}}} \quad (206)$$

C++ Implementation:

```
// Lines 894-913: Optimal scrubbing interval calculation
static double optimal_scrub_interval(
    double seu_rate_per_bit_per_sec,
    size_t num_bits,
    int correction_capability) {

    if (seu_rate_per_bit_per_sec <= 0 || num_bits == 0) {
        return 3600.0; // Default 1 hour
    }
}
```

```

// Expected errors per second across all bits
double errors_per_sec = seu_rate_per_bit_per_sec
    * static_cast<double>(num_bits);

if (errors_per_sec <= 0) {
    return 3600.0;
}

// Time until errors exceed correction capability
// Safety factor kappa = 0.5 (scrub at 50% of limit)
double max_interval = static_cast<double>(correction_capability)
    / errors_per_sec;

return max_interval * 0.5; // Apply safety factor
}

```

Example: With RS-16 ($t = 8$), 1M bits, SEU rate 5×10^{-5} errors/bit/day:

$$\lambda = 5 \times 10^{-5} \times 10^6 / 86400 = 5.79 \times 10^{-4} \text{ errors/s} \quad (207)$$

$$T_{\max} = 8 / (5.79 \times 10^{-4}) = 13,824 \text{ s} \quad (208)$$

$$T_{\text{scrub}} = 0.5 \times 13,824 = 6,912 \text{ s} \approx 1.92 \text{ hr} \quad (209)$$

12.11 Complete Decoding Pipeline Example

The following demonstrates the full RS decoding pipeline as implemented:

```

// Complete RS error correction pipeline
// (galois_field.hpp, lines 448-656)

std::optional<std::vector<element_t>> rs_correct_errors(
    const std::vector<element_t>& msg, uint8_t nsym) const {

    // STEP 1: Calculate syndromes S_0 to S_(nsym-1)
    auto syndromes = rs_calc_syndromes(msg, nsym);

    // STEP 2: Check if message has errors
    bool has_errors = false;
    for (size_t i = 1; i < nsym; ++i) {
        if (syndromes[i] != 0) {
            has_errors = true;
            break;
        }
    }
    if (!has_errors) return msg; // No errors!

    // STEP 3: Try Peterson single-error decoder (O(n))
    if (syndromes[1] != 0 && syndromes[2] != 0) {
        // Error location: X = S_2/S_1
        element_t ratio = divide(syndromes[2], syndromes[1]);
        // ... find position k where alpha^k = ratio
        // If valid single error, correct and return
    }

    // STEP 4: Try brute-force 2-error decoder (O(n^2))
    if (nsym >= 4 && msg.size() <= 32) {
        // Try all position pairs, solve 2x2 system
        // Verify with all syndromes
    }
}

```

```

// STEP 5: Try brute-force 3-error decoder (O(n^3))
if (nsym >= 6 && msg.size() <= 16) {
    // Try all position triplets, solve 3x3 system
}

// STEP 6: Fall back to Berlekamp-Massey (O(t^2))
auto [err_loc, err_eval] = rs_find_error_locator(syndromes, nsym);

// STEP 7: Chien search to find error positions (O(n*t))
auto err_pos = rs_find_errors(err_loc, msg.size());

if (err_pos.empty()) {
    return std::nullopt; // Uncorrectable!
}

// STEP 8: Forney algorithm for error magnitudes (O(t^2))
return rs_correct_errors_at_positions(msg, err_pos,
                                      err_loc, err_eval);
}

```

12.12 Summary: Math-to-Code Translation Principles

Table 22: Mathematical Notation to C++ Translation

Math Notation	C++ Implementation	Notes
$a \oplus b$	<code>a ^ b</code>	XOR operator
$a \cdot b$ in GF	<code>multiply(a, b)</code>	Via log tables
a/b in GF	<code>divide(a, b)</code>	$a \cdot b^{-1}$
α^i	<code>exp_table[i]</code>	Precomputed
$\log_\alpha(x)$	<code>log_table[x]</code>	Precomputed
$\sum$	for loop with <code>add()</code>	Accumulate
$\prod$	for loop with <code>multiply()</code>	Accumulate
$\exp(-x)$	<code>std::exp(-x)</code>	Standard library
x^s	<code>std::pow(x, s)</code>	Standard library

Key Design Decisions:

1. **Lookup Tables:** GF multiplication/division converted to table lookups for $O(1)$ operations
2. **Template Metaprogramming:** Field parameters (`m`, `Poly`) are compile-time constants
3. **Progressive Decoding:** Simple decoders tried first before expensive BM algorithm
4. **Verification:** All corrections verified by recomputing syndromes
5. **Type Safety:** `static_assert` enforces constraints at compile time

13 Template Metaprogramming and Zero-Overhead Abstractions

RadML extensively uses C++ template metaprogramming to achieve **zero-overhead abstractions**—high-level, type-safe interfaces that compile down to code as efficient as hand-written C. This section explains the compile-time vs. runtime tradeoffs and how templates address the overhead challenges inherent in radiation protection.

13.1 The Overhead Challenge

Radiation protection inherently adds overhead:

- **Memory:** ECC symbols, redundant copies (TMR), checksums
 - **Computation:** Encoding, decoding, syndrome calculation, voting
 - **Latency:** Protection checks on every memory access
- Goal:** Minimize these overheads while maintaining correctness guarantees.

13.2 Compile-Time vs. Runtime: Key Distinction

Table 23: Compile-Time vs. Runtime Computation

Aspect	Compile-Time	Runtime
When executed	During compilation	During program execution
Cost to user	Zero (happens once)	Every execution
Flexibility	Fixed at compile	Can change dynamically
Error detection	Compile errors	Runtime exceptions
Binary size	May increase	No direct impact
C++ mechanism	Templates, <code>constexpr</code>	Regular functions

13.3 Template Parameters as Compile-Time Constants

13.3.1 Galois Field: Compile-Time Field Selection

Problem: Different applications need different field sizes (GF(16), GF(256), GF(1024)).

Runtime Approach (Slow):

```
// BAD: Runtime field selection - branches on every operation
class GaloisField {
    int m; // Field order, set at runtime

    uint16_t multiply(uint16_t a, uint16_t b) {
        if (m == 4) {
            // GF(16) multiplication
        } else if (m == 8) {
            // GF(256) multiplication
        } else if (m == 10) {
            // GF(1024) multiplication
        }
        // Branch misprediction penalty: 10-20 cycles each!
    }
};
```

Compile-Time Approach (Fast):

```
// GOOD: Template parameter - decided at compile time
template <uint8_t m, uint16_t Poly>
class GaloisField {
    // m and Poly are compile-time constants
    static constexpr uint32_t field_size = (1ULL << m);
    static constexpr uint16_t primitive_poly = Poly;

    // Element type chosen at compile time based on m
    using element_t = std::conditional_t<m <= 8, uint8_t, uint16_t>;

    element_t multiply(element_t a, element_t b) const {
        // No branches! Compiler generates specialized code for this m
        return exp_table[(log_table[a] + log_table[b])
                        % (field_size - 1)];
    }
};

// Instantiations - compiler generates 3 different classes
using GF16 = GaloisField<4, 0x13>; // 16 elements
using GF256 = GaloisField<8, 0x11d>; // 256 elements
using GF1024 = GaloisField<10, 0x409>; // 1024 elements
```

What the Compiler Does:

1. Sees GaloisField<8, 0x11d>
2. Substitutes m=8, Poly=0x11d everywhere
3. Computes field_size = 256 at compile time
4. Selects element_t = uint8_t at compile time
5. Generates optimized code with no conditionals

13.3.2 Reed-Solomon: Compile-Time ECC Configuration

Problem: Different protection levels need different ECC symbol counts.

```
// Template parameters determine RS configuration at compile time
template <typename T, uint8_t SymbolSize = 8, uint8_t ECCSymbols = 8>
class AdvancedReedSolomon {
    // Compile-time assertions - caught during compilation, not runtime!
    static_assert(std::is_trivially_copyable_v<T>,
                  "T must be trivially copyable for memcpy safety");
    static_assert(SymbolSize > 0 && SymbolSize <= 16,
                  "Symbol size must be 1-16 bits");
    static_assert(ECCSymbols > 0,
                  "Must have at least 1 ECC symbol");

    // All these are computed at COMPILE TIME
    static constexpr size_t data_size = sizeof(T);
    static constexpr size_t symbol_bytes = (SymbolSize <= 8) ? 1 : 2;
    static constexpr size_t data_symbols =
        data_size / symbol_bytes + ((data_size % symbol_bytes) ? 1 : 0);
    static constexpr size_t total_symbols = data_symbols + ECCSymbols;
    static constexpr size_t total_size = total_symbols * symbol_bytes;

    // Correction capability known at compile time
```

```
constexpr size_t correction_capability() const {
    return ECCSymbols / 2; // Computed at compile time!
}
};
```

Benefits:

- `sizeof(AdvancedReedSolomon<float, 8, 8>)` is known at compile time
- Memory allocation can be optimized (stack vs heap)
- Compiler can unroll loops with known bounds
- Invalid configurations caught at compile time, not runtime

13.4 constexpr: Compile-Time Function Evaluation

`constexpr` functions can be evaluated at compile time when given constant arguments:

```
// Compile-time overhead calculation
template <typename T, uint8_t SymbolSize, uint8_t ECCSymbols>
class AdvancedReedSolomon {
    // constexpr member function -- evaluable at compile time with constexpr object
    constexpr double overhead_percent() const {
        return (static_cast<double>(total_size) / data_size - 1.0) * 100.0;
    }
};

// Usage -- overhead_percent() is a non-static member, so needs an instance:
constexpr RS8Bit8Sym<float> rs_instance{};
constexpr double overhead = rs_instance.overhead_percent();
// overhead = 200.0, computed by compiler, not at runtime!
```

13.4.1 Compile-Time Table Generation

Lookup tables can be generated at compile time:

```
// C++17: Compile-time table generation
template <uint8_t m, uint16_t Poly>
class GaloisField {
    // Tables as class members - initialized once per instantiation
    std::array<element_t, field_size> exp_table;
    std::array<element_t, field_size> log_table;

    // With C++20 constexpr, this could be fully compile-time:
    // static constexpr auto exp_table = generate_exp_table();

    // Current implementation: initialized in constructor (runtime)
    // but only ONCE per GaloisField instantiation
    GaloisField() {
        initialize_tables(); // Called once, tables reused
    }
};

// Even better: tables are typically static, so initialized once globally
// not once per object
```


13.5 Type Traits and SFINAE for Zero-Cost Type Safety

13.5.1 Compile-Time Type Checking

```
// Type trait to check if a type can be protected
template <typename T>
struct is_protectable {
    static constexpr bool value =
        std::is_trivially_copyable_v<T> && // Can memcpy safely
        (std::is_arithmetic_v<T> ||        // int, float, double
         std::is_enum_v<T> ||              // enum types
         std::is_pointer_v<T>);           // raw pointers
};

// Usage with if constexpr (C++17) - branch resolved at compile time
template <typename U>
U protect_value(const U& value, float criticality) {
    // This branch is resolved at COMPILE TIME
    // Only one branch exists in the generated code
    if constexpr (!is_protectable<U>::value) {
        return value; // Can't protect, return as-is
    } else {
        // Apply protection based on level
        // ... protection logic ...
    }
}
```

Why This Matters:

- if `constexpr` eliminates the unused branch entirely
- No runtime type checking overhead
- Compiler error if trying to protect non-copyable type

13.5.2 Conditional Type Selection

```
// Select GF type based on symbol size at compile time
template <typename T, uint8_t SymbolSize, uint8_t ECCSymbols>
class AdvancedReedSolomon {
    // Compile-time type selection - no runtime switch
    using GF = std::conditional_t<
        SymbolSize == 8, GF256,          // 8-bit symbols -> GF(256)
        std::conditional_t<
            SymbolSize == 4, GF16,        // 4-bit symbols -> GF(16)
            GF1024,                       // Otherwise -> GF(1024)
        >
    >;

    GF field_; // The right field type is baked in at compile time
};
```

13.6 Template Specialization for Algorithm Selection

Illustrative example of how template specialization could select algorithms (the actual implementation uses `AdaptiveProtectionLevel` and runtime selection, not this exact pattern):

```

// Primary template: general case
template <typename T, size_t N = sizeof(T)>
struct ProtectionStrategy {
    static void protect(T& value) {
        // Default: Reed-Solomon for larger types
        RS8Bit8Sym<T> rs;
        auto encoded = rs.encode(value);
        // ... store encoded ...
    }
};

// Specialization for small types (1-2 bytes)
template <typename T>
struct ProtectionStrategy<T, 1> {
    static void protect(T& value) {
        // Small types: Hamming code is more efficient
        // Less overhead for small data
        auto encoded = hamming_encode_byte(static_cast<uint8_t>(value));
        // ...
    }
};

// Specialization for 4-byte types (common: float, int32)
template <typename T>
struct ProtectionStrategy<T, 4> {
    static void protect(T& value) {
        // Optimized path for 32-bit values
        // Can use specialized 32-bit ECC
    }
};

// Usage - compiler selects the right specialization
ProtectionStrategy<uint8_t>::protect(byte_val); // Uses Hamming
ProtectionStrategy<float>::protect(float_val); // Uses 4-byte optimized
ProtectionStrategy<double>::protect(double_val); // Uses RS (8 bytes)

```

13.7 Inline and Link-Time Optimization

13.7.1 Forced Inlining for Hot Paths

```

template <uint8_t m, uint16_t Poly>
class GaloisField {
    // These are called millions of times - MUST be inlined
    // 'const' enables more optimizations

    // Addition: single XOR instruction when inlined
    constexpr element_t add(element_t a, element_t b) const {
        return a ^ b;
    }

    // Multiplication: 2-3 memory loads + 1 modulo when inlined
    element_t multiply(element_t a, element_t b) const {
        if (a == 0 || b == 0) return 0;
        return exp_table[(log_table[a] + log_table[b])
                          % (field_size - 1)];
    }
};

```

Assembly Output (x86-64, -O3):

```
; add(a, b) compiles to single instruction:
xor    eax, edx    ; a ^ b

; multiply(a, b) compiles to ~10 instructions:
test   edi, edi    ; if (a == 0)
je     .Lzero
test   esi, esi    ; if (b == 0)
je     .Lzero
movzx  eax, BYTE PTR log_table[rdi]
movzx  edx, BYTE PTR log_table[rsi]
add    eax, edx
; ... modulo and exp_table lookup ...
```

13.8 Memory Layout Optimization**13.8.1 Compile-Time Size Calculation**

```
template <typename T, uint8_t SymbolSize = 8, uint8_t ECCSymbols = 8>
class AdvancedReedSolomon {
    // All sizes known at compile time
    static constexpr size_t data_size = sizeof(T);
    static constexpr size_t symbol_bytes = (SymbolSize <= 8) ? 1 : 2;
    static constexpr size_t data_symbols =
        data_size / symbol_bytes + ((data_size % symbol_bytes) ? 1 : 0);
    static constexpr size_t total_symbols = data_symbols + ECCSymbols;
    static constexpr size_t total_size = total_symbols * symbol_bytes;

    GF field_; // Galois field instance (owns exp/log tables)
    std::vector<element_t> generator_poly_; // Generator polynomial
};

// sizeof(AdvancedReedSolomon<float, 8, 8>) is known at compile time
// Allows efficient object placement and cache optimization
```

13.8.2 Cache-Friendly Data Layout

```
// TMR with compile-time layout
template <typename T>
class TMR {
    static constexpr size_t num_copies_ = 3;

    // Contiguous array - cache friendly
    // Size known at compile time: 3 * sizeof(T)
    T copies_[num_copies_];

    // Majority vote accesses all 3 copies
    // They're adjacent in memory -> good cache behavior
    T performMajorityVote() const {
        // copies_[0], copies_[1], copies_[2] are contiguous
        // Single cache line for small T
        if (copies_[0] == copies_[1]) return copies_[0];
        if (copies_[0] == copies_[2]) return copies_[0];
        if (copies_[1] == copies_[2]) return copies_[1];
        return copies_[0]; // All different - uncorrectable
    }
}
```

```
};
```

13.9 Overhead Comparison: Templates vs. Runtime Polymorphism

Table 24: Performance Comparison: Templates vs. Virtual Functions

Metric	Template	Virtual Function
Function call overhead	0 (inlined)	10-20 cycles (vtable lookup)
Memory per object	0 extra	8 bytes (vptr)
Branch prediction	Perfect	Often mispredicted
Compiler optimization	Full (sees all code)	Limited (across boundary)
Code size	May increase	Shared code
Flexibility	Compile-time only	Runtime selection

RadML Design Choice: Use templates for hot paths (GF operations, encoding), runtime polymorphism only for configuration/setup.

13.10 Practical Example: Complete Protection Pipeline

Illustrative example showing how compile-time type selection works in the pipeline. The actual implementation uses `RSProtectedValue` in `multi_bit_protection.hpp`:

```
// Conceptual illustration of compile-time protection pipeline
template <typename T>
class RadiationProtectedValue {
    using RS = std::conditional_t<
        sizeof(T) <= 2,
        AdvancedReedSolomon<T, 8, 4>, // Small: RS(n,k) with t=2
        AdvancedReedSolomon<T, 8, 8> // Large: RS(n,k) with t=4
    >;

    RS codec_;
    std::vector<uint8_t> encoded_;

public:
    void store(const T& value) {
        encoded_ = codec_.encode(value);
    }

    std::optional<T> load() const {
        return codec_.decode(encoded_);
    }
};

// Usage
RadiationProtectedValue<float> protected_weight;
protected_weight.store(3.14159f);

// This is evaluated at COMPILE TIME:
static_assert(RadiationProtectedValue<float>::overhead() < 300.0,
    "Protection overhead exceeds budget");
```

13.11 Summary: Template Metaprogramming Benefits

Key Principle: “Don’t pay for what you don’t use” (C++ design philosophy)

Table 25: Compile-Time vs. Runtime in RadML

Component	Compile-Time	Runtime
GF field size	✓(template param)	
GF primitive polynomial	✓(template param)	
GF element type	✓(conditional_t)	
RS ECC symbol count	✓(template param)	
RS correction capability	✓(constexpr)	
Memory overhead calculation	✓(constexpr)	
Type safety checks	✓(static_assert)	
Protection level selection		✓(environment-based)
Actual encoding/decoding		✓(data-dependent)
Error detection		✓(runtime syndromes)

- Configuration decisions → compile time (zero runtime cost)
- Data-dependent operations → runtime (unavoidable)
- Type checking → compile time (catch errors early)
- Algorithm selection → compile time via specialization

Result: RadML’s protection abstractions compile to code nearly as efficient as hand-optimized C, while providing type safety and configurability that raw C cannot match.

14 Framework Architecture Diagrams

This section presents visual diagrams showing how the RadML components are structured and interact in the actual codebase.

14.1 High-Level System Architecture

14.2 Protection Pipeline Flow

14.3 Protection Level Selection

14.4 Reed-Solomon Pipeline

14.5 GF(256) Multiplication

14.6 TMR Voting

14.7 Memory Layout with ECC

14.8 Radiation Environment Mapping

15 Defense in Depth: Complete Protection Stack

RadML implements a **defense-in-depth** strategy where multiple independent protection mechanisms work together. If one layer fails, others provide backup protection. This section describes all protection layers and how they integrate.

15.1 Defense Layers Overview

Table 26: Defense-in-Depth Protection Layers

Layer	Component	Protection Type	File Location
1. Memory Placement	RadiationMappedAllocator	Physical shielding	memory/
2. Memory Scrubbing	MemoryScrubber	Proactive detection	core/memory/
3. Data Encoding	ECC (Hamming/RS)	Error correction	neural/
4. Spatial Redundancy	TMR variants	Voting/masking	tmr/
5. Temporal Redundancy	TemporalRedundancy	Time-based voting	tmr/
6. Hybrid Redundancy	HybridRedundancy	Spatial + Temporal	tmr/
7. Checkpointing	CheckpointManager	State recovery	core/recovery/
8. Error Tracking	RadiationErrorTracker	Trend analysis	core/runtime/
9. Power Management	PowerAwareProtection	Resource optimization	power/
10. Selective Hardening	SelectiveHardening	Cost-effective protection	neural/

15.2 Layer 1: Radiation-Aware Memory Placement

File: memory/radiation_mapped_allocator.hpp

The first line of defense is physical: placing critical data in naturally shielded memory regions.

```
// Memory zones with different radiation vulnerability
enum class Level {
    HIGHLY_SHIELDED,    // SEU rate: 1e-11 events/bit-day
    MODERATELY_SHIELDED, // SEU rate: 1e-9
    LIGHTLY_SHIELDED,   // SEU rate: 1e-8
    UNSHIELDED          // SEU rate: 1e-7
};

// Criticality-based allocation
enum class DataCriticality {
    MISSION_CRITICAL,    // -> HIGHLY_SHIELDED + TMR
    HIGHLY_IMPORTANT,    // -> MODERATELY_SHIELDED + ECC
    MODERATELY_IMPORTANT, // -> LIGHTLY_SHIELDED + CRC
    LOW_IMPORTANCE       // -> UNSHIELDED + canary
};
```

Key Principle: The allocator automatically places MISSION_CRITICAL data in regions with lowest SEU rates, reducing the load on software protection mechanisms.

15.3 Layer 2: Proactive Memory Scrubbing

File: core/memory/memory_scrubber.hpp

Memory scrubbing periodically scans protected regions to detect and correct errors before they accumulate into uncorrectable multi-bit errors.

```
class MemoryScrubber {
    // Scrub interval based on SEU rate (Equation 32)
    explicit MemoryScrubber(unsigned long scrub_interval_ms = 1000);
```

```

// Register memory regions for periodic verification
size_t registerMemoryRegion(T* ptr, size_t size,
    std::function<void(T*, size_t)> scrub_function);

void start(); // Begin background scrubbing thread
};

```

Integration with ECC: The scrubber calls ECC decode on each region. If syndromes are non-zero, corrections are applied immediately.

15.4 Layer 3: Error-Correcting Codes (ECC)

Files: `neural/galois_field.hpp`, `neural/advanced_reed_solomon.hpp`, `neural/adaptive_protection.hpp`

This layer provides the core error correction capability (detailed in Sections 3-4):

- **Parity** – Detects 1-bit errors (12.5% overhead)
- **Hamming(7,4)** – Corrects 1-bit, detects 2-bit (75% overhead)
- **RS-8** – Corrects up to 4 symbol errors (200% overhead)
- **RS-16** – Corrects up to 8 symbol errors (400% overhead)

ECC is the workhorse of the protection stack, handling the majority of single-event upsets.

15.5 Layer 4: Spatial Redundancy (TMR Variants)

Files: `tmr/tmr.hpp`, `tmr/enhanced_tmr.hpp`, `tmr/health_weighted_tmr.hpp`, `tmr/enhanced_stuck_bit_tmr.hpp`

TMR provides spatial redundancy by maintaining three copies and using majority voting:

Table 27: TMR Variants in RadML

Variant	Enhancement	Use Case
Basic TMR	Simple majority vote	General protection
Enhanced TMR	CRC32 checksums	Silent data corruption
Health-Weighted TMR	Per-copy reliability scores	Long missions
Stuck-Bit TMR	Bit-level error tracking	Flash memory
Approximate TMR	Fuzzy voting for floats	Neural network weights

```

// Health-weighted voting (tmr/health_weighted_tmr.hpp)
template <typename T>
class HealthWeightedTMR {
    std::array<T, 3> copies_;
    std::array<double, 3> health_scores_; // 0.0-1.0

    T get() const {
        // Weight votes by health scores
        // Prefer copies with higher reliability history
    }
};

```


15.6 Layer 5: Temporal Redundancy

File: `tmr/temporal_redundancy.hpp`

Temporal redundancy executes operations multiple times with delays to avoid correlated transient faults:

```
template <typename T, typename ResultType>
class TemporalRedundancy {
    size_t num_executions_;           // Default: 3
    std::chrono::milliseconds delay_; // Default: 10ms

    ResultType execute(const T& data,
                       std::function<ResultType(const T&)> op) {
        std::vector<ResultType> results;
        for (size_t i = 0; i < num_executions_; ++i) {
            results.push_back(op(data));
            std::this_thread::sleep_for(delay_); // De-correlate
        }
        return findMostCommonResult(results); // Temporal voting
    }
};
```

Key Insight: The delay between executions ensures that a transient radiation event affecting one execution is unlikely to affect subsequent ones.

15.7 Layer 6: Hybrid Redundancy

File: `tmr/hybrid_redundancy.hpp`

Hybrid redundancy combines spatial (TMR) and temporal protection with checkpointing:

```
template <typename T>
class HybridRedundancy {
    EnhancedStuckBitTMR<T> tmr_;
    TemporalRedundancy<T, T> temporal_;
    CheckpointManager<T> checkpoint_mgr_;

    T get() const {
        // Temporal redundancy wraps TMR get
        return temporal_.execute(tmr_, [](auto& t) { return t.get(); });
    }

    bool repair() {
        tmr_.repair();
        T before = tmr_.get();
        T after = temporal_.execute(tmr_, ...);
        if (before != after) return rollback(); // Use checkpoint
        return true;
    }
};
```

Fallback Chain: TMR → Temporal Validation → Checkpoint Rollback

15.8 Layer 7: Checkpoint and Recovery

File: `core/recovery/checkpoint_manager.hpp`

Checkpointing provides the last line of defense—if all other protections fail, the system can rollback to a known-good state:

```
template <typename T>
class CheckpointManager {
    size_t max_checkpoints_; // Rolling buffer (default: 5)
    std::chrono::seconds checkpoint_interval_;
```

```

    void createCheckpoint(const T& data, uint64_t version);
    bool rollbackToValid(T& data,
        std::function<bool(const T&)> validator);
};

```

Validation-Based Rollback: The `rollbackToValid` method searches for the most recent checkpoint that passes a user-defined validation function (e.g., CRC check, bounds check).

15.9 Layer 8: Runtime Error Tracking

File: `core/runtime/error_tracker.hpp`

The error tracker provides real-time visibility into radiation effects and enables adaptive responses:

```

class RadiationErrorTracker {
    std::atomic<uint64_t> error_count;
    std::atomic<float> current_error_rate;
    std::array<std::atomic<uint32_t>, N> pattern_counts;

    void recordError(FaultPattern pattern);
    float getErrorRate() const;

    // Pattern analysis for adaptive protection
    std::map<FaultPattern, double> getPatternDistribution();
};

```

Feedback Loop: Error rate trends trigger protection level adjustments (see Layer 10).

15.10 Layer 9: Power-Aware Protection

File: `power/power_aware_protection.hpp`

In spacecraft, power is a constrained resource. This layer optimizes protection levels based on power budget:

```

enum class PowerState {
    EMERGENCY,           // 20% budget - critical systems only
    LOW_POWER,           // 40% budget - essential systems
    NOMINAL,             // 70% budget - normal operation
    SCIENCE_OPERATION,   // 90% budget - full science
    PEAK_PERFORMANCE    // 100% budget - maximum protection
};

class PowerAwareProtection {
    void optimizeProtectionLevels(); // Solve optimization problem
    // Maximize sum(criticality_i * protection_i)
    // Subject to: sum(power_i) <= power_budget
};

```

Trade-off: In EMERGENCY mode, protection is reduced on less critical components to maintain power for mission-critical systems.

15.11 Layer 10: Selective Hardening

File: `neural/selective_hardening.hpp`

Not all neural network components are equally sensitive to errors. Selective hardening applies heavy protection only where needed:

```

enum class HardeningStrategy {
    PROTECT_ALL,         // Protect all components equally
    ALL_LAYERS,          // Protect all layers
    CRITICAL_LAYERS,     // Only layers on critical paths
    WEIGHT_THRESHOLD,    // Only weights > threshold
};

```

```

    FIXED_THRESHOLD,    // Above fixed criticality threshold
    RESOURCE_CONSTRAINED, // Within resource budget
    ADAPTIVE_RUNTIME,    // Adapt at runtime by error rates
    ADAPTIVE,            // Based on sensitivity analysis
    LAYERWISE_IMPORTANCE // Output layers more critical
};

class SelectiveHardening {
    // Analyze which layers impact output most
    SensitivityResult analyzeSensitivity(const Network& net);

    // Apply appropriate protection per layer
    void applyProtection(Layer& layer, ProtectionLevel level);
};

```

Sensitivity Analysis (neural/sensitivity_analysis.hpp): Computes how much each layer’s errors propagate to the output, allowing targeted protection.

15.12 Defense-in-Depth Integration Diagram

15.13 How Layers Work Together: Example Scenario

Consider a critical neural network weight stored in RadML during a solar particle event:

1. **Layer 1 (Memory Placement):** Weight is allocated in **HIGHLY_SHIELDED** region, reducing base SEU rate by $10,000\times$ (from 10^{-7} to 10^{-11} events/bit-day).
2. **Layer 2 (Scrubbing):** Every 100ms, the scrubber verifies the weight’s ECC syndrome.
3. **Layer 3 (ECC):** A cosmic ray causes a 2-bit error. RS-8 detects and corrects it.
4. **Layer 4 (TMR):** During high radiation (SAA), the weight is replicated $3\times$. A second hit corrupts one copy—majority voting masks it.
5. **Layer 5 (Temporal):** Inference runs $3\times$ with 10ms delays. Transient error in one run is outvoted.
6. **Layer 7 (Checkpoint):** Every 60s, a checkpoint is saved. If corruption is detected, rollback occurs.
7. **Layer 8 (Tracking):** Error rate spikes trigger Layer 10 to increase protection on critical paths.
8. **Layer 9 (Power):** If power drops to **LOW_POWER**, protection on non-critical layers is reduced to maintain critical protection.

Result: Multiple independent failures would need to occur simultaneously to corrupt the final output—the hallmark of defense-in-depth.

15.14 Physics-Driven Protection Integration

File: tmr/physics_driven_protection.hpp

The framework integrates NASA physics models to drive protection decisions:

```

class PhysicsModels {
    // Temperature affects SEU threshold (Arrhenius equation)
    static double calculateTemperatureCorrectedThreshold(
        double base_threshold, double temperature_K);

    // Mechanical stress increases vulnerability

```

```

static double calculateMechanicalLoadFactor(
    double stress_MPa, double yield_MPa, double dose);

// Synergistic effects (temperature + stress)
static double calculateSynergyFactor(...);
};

```

These models are used by `AdaptiveProtection` to adjust ECC levels based on real-time environment conditions (temperature, orbital position, solar activity).

15.15 Validation Through Fault Injection

File: `testing/fault_injection.hpp`

The defense-in-depth stack is validated through systematic fault injection:

```

struct FaultInjectionResult {
    int total_injected_faults;
    int detected_faults;
    int corrected_faults;
    double correction_rate; // Should be > 99.9% for critical data
};

class SystematicFaultInjector {
    // Inject random bit flips, burst errors, stuck bits
    void injectFaults(Network& net, FaultPattern pattern);

    // Run full test campaign
    std::vector<FaultInjectionResult> runCampaign();
};

```

15.16 Summary: Defense-in-Depth Effectiveness

Table 28: Protection Effectiveness by Layer

Layer	Handles	Overhead	Latency
Memory Placement	Reduces exposure	0%	0
Memory Scrubbing	Accumulating errors	1-5%	Background
ECC (RS-8)	1-4 symbol errors	200%	$O(n)$
TMR	Single copy corruption	200%	$O(1)$
Temporal	Transient faults	200% (time)	$3 \times$ latency
Checkpointing	Catastrophic failure	5-10% storage	Recovery time
Error Tracking	Trend detection	<1%	Real-time

Key Insight: Each layer addresses different failure modes. ECC handles bit errors efficiently but can't handle multi-copy corruption. TMR handles copy corruption but has high overhead. Checkpointing handles catastrophic failures but with recovery latency. Together, they provide comprehensive protection.

16 Why RadML Works: Integration Architecture

This section explains the **design philosophy** behind RadML—why each component exists and how they work together to provide defense-in-depth protection.

16.1 The Core Integration Pipeline

End-to-End Protection Flow

Physics → SEU Rate → Protection Level → Resource Allocation → EC-C/TMR Selection

1. **Environment Sensing:** Orbital position, solar activity, SAA passage
2. **Flux Calculation:** NASA models (GCR, AP-8, AE-8) → $\Phi(E)$
3. **Cross-Section:** Weibull (heavy ions) + Bendel (protons) → $\sigma(E)$
4. **SEU Rate:** $R_{\text{SEU}} = \int \Phi(E) \cdot \sigma(E) dE$
5. **Protection Decision:** Map R_{SEU} to protection level
6. **Resource Allocation:** Distribute protection budget across layers
7. **Apply Protection:** Select ECC scheme and TMR variant

16.2 Why Physics-Driven Protection Works

16.2.1 The Problem with Static Protection

Static protection (always use TMR, always use RS-8) fails because:

- **Waste in benign environments:** 200% overhead when SEU rate is 10^{-8}
- **Insufficient in harsh environments:** RS-8 can't handle SAA burst errors
- **No awareness of criticality:** Output layer matters more than hidden layers

16.2.2 How RadML Solves This

```
// In adaptive_protection.hpp:443
void adapt_to_environment() {
    double current_seu_rate = get_current_seu_probability();

    // Physics drives the decision
    if (current_seu_rate > 1e-3) { // SAA, solar storm
        protection_level_ = VERY_HIGH; // RS-16 (8 byte correction)
    } else if (current_seu_rate > 1e-4) { // Nominal orbit
        protection_level_ = HIGH; // RS-8 (4 byte correction)
    } else if (current_seu_rate > 1e-5) { // Quiet conditions
        protection_level_ = MODERATE; // Hamming (1 bit/nibble)
    } else { // Deep space
        protection_level_ = MINIMAL; // Parity only
    }
}
```

Why this works:

1. SEU rate is a **measured physical quantity**, not a guess
2. Thresholds (10^{-3} , 10^{-4} , 10^{-5}) calibrated to ECC capabilities
3. Protection overhead scales with actual risk

16.3 Why Sensitivity-Based Resource Allocation Works

Not all neural network parameters are equally important. RadML allocates protection resources based on:

$$\text{Importance}_i = \text{Sensitivity}_i \times \text{EnvSeverity} \times \text{PositionFactor}_i \quad (210)$$

```
// In physics_driven_protection.hpp:158-162
for (int i = 0; i < num_layers; i++) {
    // Earlier layers propagate errors forward -> more critical
    double position_factor = 1.0 - (0.5 * i / num_layers);

    importance_factors[i] = layer_sensitivities[i]
        * env_severity
        * position_factor;
}

// Allocate resources proportionally
optimal_allocation[i] = (importance_factors[i] / sum_importance)
    * total_protection_resources;
```

Why this works:

- **Gradient-based sensitivity:** Weights with large gradients affect output more
- **Position factor:** Early layer errors propagate through entire network
- **Constrained optimization:** Fixed budget allocated where it matters most

16.4 Why Quantum Corrections Improve Accuracy

Classical Weibull/Bendel models are *empirical fits*. Quantum corrections capture:

Quantum Effects Not in Classical Models

1. **Quantum Tunneling:** Electrons tunnel through barriers → lower effective Q_{crit}
2. **Temperature Dependence:** Thermal phonons assist charge transport
3. **Size Effects:** <100nm features have quantum confinement → increased sensitivity
4. **Defect Clustering:** QFT predicts correlated defect formation → MBU patterns

```
// In quantum_enhanced_radiation.cpp:140-151
double quantum_size_factor = 1.0;
if (feature_size_nm < 100.0) {
    // Quantum confinement energy
    double confinement_energy = (HBAR^2 * PI^2) /
        (2 * m_eff * (feature_size_nm * 1e-9)^2);

    // Higher confinement -> more sensitivity
    quantum_size_factor = 1.0 + confinement_energy / bandgap;
}
return base_sensitivity * quantum_size_factor;
```

Why this works:

- Classical models calibrated at 90nm+ don't predict 7nm behavior
- Quantum corrections scale $\propto 1/L^2$ (confinement energy)
- Validated against experimental SEU data from modern processes

16.5 Why the Defense-in-Depth Stack Works

Each protection layer handles **different failure modes**:

Table 29: Failure Mode Coverage Matrix

Failure Mode	ECC	TMR	Scrub	Ckpt	Track
Single bit flip	✓✓	✓	✓		
Multi-bit upset	✓		✓		
Copy corruption		✓✓			
Stuck bit		✓		✓	✓
Transient glitch				✓	
Accumulating errors			✓✓		✓
Catastrophic failure				✓✓	

Why this works:

- No single protection handles all failures
- Layers are **orthogonal**: ECC fixes bits, TMR fixes copies, Ckpt fixes state
- **Overlapping coverage**: A failure that escapes ECC gets caught by scrubbing

16.6 Why TMR Variants Exist

Different scenarios need different TMR strategies.

Implementation note: The `tmr/` directory contains two parallel class hierarchies:

1. **Storage classes** (`tmr.hpp`, `enhanced_tmr.hpp`, etc.): Protect a stored T value with three internal copies.
2. **Strategy classes** (`adaptive_protection_impl.hpp`): Execute an operation three times and vote on results.

Some names (e.g., `EnhancedTMR`, `HealthWeightedTMR`, `HybridRedundancy`) exist in both hierarchies with different behavior.

```
// Protection level selection based on environment
// In tmr/adaptive_protection_impl.hpp:
// TMRStrategyFactory::calculateOptimalProtectionLevel()
double protection_need = radiation_factor * criticality;
if (env.saa_region) protection_need *= 2.0;
if (env.solar_activity > 0.7) protection_need *= 1.5;

if (protection_need > 5.0 || criticality > 0.9) {
    level = HYBRID_REDUNDANCY; // Maximum protection
} else if (protection_need > 2.0 || criticality > 0.7) {
    level = HEALTH_WEIGHTED_TMR; // Tracks copy reliability
} else if (protection_need > 1.0 || criticality > 0.5) {
    level = ENHANCED_TMR; // CRC + voting
} else if (protection_need > 0.5) {
    level = BASIC_TMR; // Simple majority
}
```

16.7 Integration Code Path Example

Complete flow from orbit to protection:

Table 30: TMR Variant Selection Logic

Variant	When to Use	Why
BasicTMR	Low radiation, simple data	Lowest overhead
EnhancedTMR	Moderate radiation	CRC detects silent corruption
HealthWeightedTMR	Varying reliability	Demotes unreliable copies
StuckBitTMR	Known stuck bits	Masks out bad locations
HybridRedundancy	Critical, long-duration	Combines spatial+temporal
ApproximateTMR	Floating point	Tolerates numerical differences

```

// 1. Create physics environment
PhysicsRadiationEnvironment::Config config;
config.altitude_km = 400.0;          // ISS orbit
config.inclination_deg = 51.6;
config.solar_modulation_mv = 650.0; // Solar cycle
config.heavy_ion_device = WeibullParameters::cmos_28nm();
PhysicsRadiationEnvironment env(config);

// 2. Get physics-based SEU rate
double seu_rate = env.get_orbit_average_seu_rate(); // ~3e-6/bit/day

// 3. Create adaptive protection with this environment
AdaptiveProtection<float> protection(env);

// 4. Protection level is automatically set based on seu_rate
// seu_rate = 3e-6 -> MODERATE (Hamming protection)

// 5. Protect critical weights
for (auto& weight : network.weights()) {
    protection.protect_value(weight); // Applies Hamming encoding
}

// 6. During SAA passage, environment updates
env.updatePosition(lat, lon, alt); // Now in SAA
seu_rate = env.get_current_seu_rate(); // ~1e-3/bit/day (1000x increase)
protection.adapt_to_environment(); // Switches to VERY_HIGH (RS-16)

// 7. Critical weights now use RS-16 (8-byte correction)

```

16.8 Performance vs Protection Trade-off

RadML's design philosophy: **Pay for protection only when needed**

$$\text{Effective Overhead} = \sum_{\text{environments}} P(\text{env}) \times \text{Overhead}(\text{env}) \quad (211)$$

Example: ISS Mission Profile

- 85% time in nominal orbit: MODERATE (75% overhead)
- 10% time in SAA: VERY_HIGH (400% overhead)
- 5% time in solar event: VERY_HIGH (400% overhead)

$$\text{Effective Overhead} = 0.85 \times 0.75 + 0.15 \times 4.0 = 1.24 = 124\% \quad (212)$$

vs. static VERY_HIGH: 400% always → **3.2x savings**

16.9 Why the Transport Equation Matters

The Boltzmann transport equation provides the **theoretical foundation** for all flux models:

$$\frac{\partial \Phi}{\partial t} + \hat{\Omega} \cdot \nabla \Phi + \Sigma_t \Phi = \int \Sigma_s \Phi' d\Omega' dE' + S \quad (213)$$

Connection to Production Models:

Why NASA models are pre-computed transport solutions:

- AP-8/AP-9: Solves transport through Van Allen belts (trapped protons)
- AE-8/AE-9: Solves transport through Van Allen belts (trapped electrons)
- GCR models: Solves transport through heliosphere + magnetosphere
- RadML uses these solutions directly → no need to re-solve transport
- `transport_equation.hpp` available for custom shielding analysis

16.10 Summary: The RadML Design Philosophy

Core Design Principles

1. **Physics-First:** Protection decisions driven by measured SEU rates, not guesses
2. **Adaptive:** Protection scales with environment—minimal in quiet periods, maximum in SAA
3. **Efficient:** Sensitivity analysis allocates resources to critical components
4. **Defense-in-Depth:** Multiple orthogonal layers catch different failure modes
5. **Validated:** NASA flux models + Weibull/Bendel cross-sections are industry standard
6. **Extensible:** Quantum corrections layer for improved accuracy at small feature sizes
7. **Zero-Overhead Abstractions:** Template metaprogramming eliminates runtime dispatch

17 Complete RadML Framework Architecture

This section provides a comprehensive inventory of all RadML components, demonstrating the full scope of the radiation-tolerant machine learning framework.

17.1 Framework Overview

RadML Architecture

RadML is a comprehensive C++ framework for radiation-tolerant machine learning, featuring:

- **8 TMR variants** across 10 header modules for redundancy-based protection
- **Reed-Solomon & Hamming** error correcting codes
- **First-principles quantum physics** models
- **NASA/ESA validated** flux and cross-section models
- **PyTorch integration** for deep learning
- **Automated architecture search** (evolutionary, grid, random)
- **Full mission simulation** infrastructure

17.2 Advanced Protection Mechanisms

17.2.1 Algorithmic Diversity

File: advanced/algorithmic_diversity.hpp

Multiple implementations of the same operation with reliability-weighted voting:

```
template <typename T, typename ResultType>
class AlgorithmicDiversity {
public:
    // Register multiple implementations with initial reliability
    void addImplementation(const std::string& name,
                          std::function<ResultType(const T&)> impl,
                          double initial_reliability = 1.0);

    // Execute all and return reliability-weighted consensus
    ResultType execute(const T& data);

    // Update reliability based on observed correctness
    void updateReliability(const std::string& name, bool success);

    // Decay-weighted reliability tracking
    // new_reliability = old * 0.9 + recent_success_rate * 0.1
};
```

Use Case: Critical computations where different algorithms may fail differently under radiation.

17.2.2 Neural Network-Based Error Prediction

File: advanced/error_prediction.hpp

ML-based detection and correction of radiation-induced errors:

```
template <typename ModelType, typename InputType, typename OutputType>
class RadiationErrorPredictor {
public:
    // Train on pairs of (corrupted, correct) outputs
    bool train(const std::vector<std::pair<InputType, OutputType>>& radiation_affected,
              const std::vector<std::pair<InputType, OutputType>>& correct);

    // Detect error probability (0.0 = likely clean, 1.0 = likely corrupted)
```

```

float detectErrorProbability(const InputType& input, const OutputType& output);

// Suggest correction if confidence exceeds threshold
std::optional<OutputType> suggestCorrection(const InputType& input,
                                           const OutputType& suspect);
};

```

17.3 Research and Architecture Optimization

17.3.1 Automated Architecture Search

File: research/auto_arch_search.hpp

Systematic search for optimal radiation-tolerant network architectures:

```

class AutoArchSearch {
public:
    // Grid search: exhaustive over all configurations
    SearchResult findOptimalArchitecture(size_t max_epochs = 50,
                                       bool use_monte_carlo = false,
                                       size_t monte_carlo_trials = 50);

    // Random search: sample configurations randomly
    SearchResult randomSearch(size_t max_iterations = 30, ...);

    // Evolutionary search: genetic algorithm optimization
    SearchResult evolutionarySearch(size_t population_size = 10,
                                   size_t generations = 10,
                                   double mutation_rate = 0.1, ...);
};

```

Search Space:

- Layer widths: {32, 64, 128, 256}
- Dropout rates: {0.3, 0.4, 0.5, 0.6, 0.7}
- Protection levels: {NONE, MINIMAL, MODERATE, HIGH, ADAPTIVE_TMR}
- Residual connections: {true, false}

17.3.2 Radiation-Tolerant Variational Autoencoder

File: research/variational_autoencoder.hpp

Production-ready VAE with radiation protection for telemetry compression:

```

template <typename T = float>
class VariationalAutoencoder {
public:
    VariationalAutoencoder(size_t input_dim, size_t latent_dim,
                          const std::vector<size_t>& hidden_dims,
                          neural::ProtectionLevel protection,
                          const VAEConfig& config);

    // Production training (epochs from VAEConfig, not a parameter)
    TrainingMetrics trainProduction(
        const std::vector<std::vector<T>>& train_data,
        const std::vector<std::vector<T>>& val_data = {});

    // Encode to latent space -- returns (mean, log_variance) pair
    std::pair<std::vector<T>, std::vector<T>> encode(
        const std::vector<T>& input, double radiation_level = 0.0);
};

```

```

// Decode from latent space
std::vector<T> decode(const std::vector<T>& latent,
                    double radiation_level = 0.0);

// Comprehensive evaluation metrics
std::unordered_map<std::string, float> evaluateComprehensive(
    const std::vector<std::vector<T>>& data);
};

```

Applications:

- Telemetry compression (3:1 ratio demonstrated)
- Anomaly detection for spacecraft health monitoring
- Synthetic data generation for training augmentation

17.3.3 Radiation-Aware Training

File: research/radiation_aware_training.hpp

Inject bit flips during training to build inherent resilience:

```

template <typename NetworkType>
class RadiationAwareTrainer {
public:
    // Train with simulated bit flips
    TrainingStats train(NetworkType& network,
                        const std::vector<float>& data,
                        const std::vector<float>& labels,
                        float bit_flip_probability = 1e-6);

    // Target critical weights for injection
    void setCriticalityMap(const std::vector<float>& criticality_scores);
};

```

17.4 PyTorch Integration

File: pytorch/pytorch_integration.hpp

Bridge between PyTorch and RadML protection mechanisms:

```

#ifdef RAD_ML_PYTORCH_ENABLED

class ProtectedTensor {
public:
    explicit ProtectedTensor(const torch::Tensor& tensor);

    torch::Tensor& tensor();
    void enable_protection(neural::ProtectionLevel level);
    void enable_tmr_protection(tmr::ProtectionLevel strategy);
    void apply_radiation_hardening();
    void validate_integrity();
};

class RadiationHardenedModule {
protected:
    virtual torch::Tensor forward_protected(torch::Tensor input) = 0;
    virtual void apply_weight_protection() = 0;
    virtual void validate_weights() = 0;
};

#endif

```

Configuration:

```

struct PyTorchConfig {
    bool enable_tmr_protection = true;
    bool enable_radiation_hardening = true;
    neural::ProtectionLevel protection_level = neural::ProtectionLevel::SELECTIVE_TMR;
    bool use_cuda_if_available = true;
    bool enable_gradient_protection = true;
    bool enable_weight_protection = true;
};
// neural::ProtectionLevel values: NONE, CHECKSUM_ONLY,
// SELECTIVE_TMR, FULL_TMR, ADAPTIVE_TMR, SPACE_OPTIMIZED

```

17.5 AI-Native Database

File: storage/ai_native_database.hpp

VAE-based intelligent compression with LMDB persistence:

```

class AINativeDatabase {
public:
    struct Config {
        std::filesystem::path db_path = "./ai_native_db";
        size_t max_db_size = 10ULL * 1024 * 1024 * 1024; // 10GB
        size_t default_latent_dim = 32;
        float max_reconstruction_error = 0.005f; // 0.5%
        bool enable_background_optimization = true;
    };

    // Async store with VAE compression
    template <typename T>
    std::future<Result<CompressionMetrics>> store_async(
        const Key& key, const std::vector<T>& data);

    // Async retrieve with VAE decompression
    template <typename T>
    std::future<Result<std::pair<std::vector<T>, CompressionMetrics>>>
        retrieve_async(const Key& key);
};

```

17.6 Mission Simulation Infrastructure

17.6.1 Full Mission Simulator

File: testing/mission_simulator.hpp

Complete mission simulation with environment transitions:

```

struct MissionProfile {
    std::string name;
    std::vector<RadiationSimulator::EnvironmentParams> environments;
    std::vector<double> environment_durations; // seconds
    std::vector<double> transition_probabilities;

    // Predefined missions: LEO, ISS, GEO, LUNAR, MARS, JUPITER
    static MissionProfile createStandard(const std::string& mission_type);
};

class MissionSimulator {
public:
    MissionSimulator(const MissionProfile& profile,
        const AdaptiveProtectionConfig& protection);

    template <typename T>
    void registerMemoryRegion(T* memory, size_t size, bool is_protected);
};

```

```

MissionStatistics runSimulation(
    std::chrono::seconds total_duration,
    std::chrono::milliseconds time_step = 1000ms,
    std::function<void(const EnvironmentParams&)> on_change = nullptr);
};

```

MissionStatistics tracks:

- Single bit flips, multi-bit upsets, latchups, transients
- Errors detected, corrected, undetected
- Time and events by environment
- TMR activations, scrubbing cycles
- Mission-critical uptime percentage

17.6.2 Physics-Based Radiation Simulator

File: `sim/physics_radiation_simulator.hpp`

Detailed space environment modeling based on NASA AE9/AP9:

```

enum class SpaceEnvironment {
    LEO, EARTH_ORBIT, MEO, GEO, LUNAR,
    MARS, MARS_ORBIT, MARS_SURFACE,
    JUPITER, EUROPA, INTERPLANETARY,
    SOLAR_PROBE, SOLAR_MINIMUM, SOLAR_MAXIMUM, SOLAR_STORM
};

enum class RadiationEffectType {
    SEU,           // Single Event Upset (1e-7/bit/day LEO)
    MBU,           // Multiple Bit Upset (2e-8/bit/day)
    SEL,           // Single Event Latchup (5e-9/bit/day)
    SET,           // Single Event Transient (2e-7/bit/day)
    SEFI,          // Single Event Functional Interrupt (1e-9/bit/day)
    TID_STUCK_BIT, // Total Ionizing Dose stuck bit (5e-10/bit/day)
    TID_THRESHOLD_SHIFT // TID threshold shift (1e-9/bit/day)
};

```

17.7 NASA/ESA Validation Protocol

File: `validation/nasa_esa_validation_protocol.hpp`

Comprehensive verification according to space standards:

```

class NASAESAVerificationProtocol {
public:
    // Standards implemented:
    // - NASA-HDBK-4002A: Mitigating In-Space Charging Effects
    // - ECSS-E-ST-10-12C: Radiation calculation methods
    // - JEDEC JESD57: SEE measurement procedures
    // - NASA/TP-2006-214373: SEE Criticality Analysis
    // - MIL-STD-883, Method 1019: Total dose test

    enum class EnvironmentModel { CREME96, OMERE, SPENVIS, AP8_AE8, SHIELDOSE };
    enum class TestEnvironment { LEO, SAA, GEO, VAN_ALLEN, LUNAR, MARS, JUPITER };

    struct StandardMetric {
        std::string name;
        double value, reference_value, threshold;
        double confidence_interval_low, confidence_interval_high;
        VerificationStatus status; // PASS, FAIL, NOT_TESTED
    };
};

```

17.8 Material Database

File: core/material_database.hpp

Comprehensive material properties for radiation simulation:

```
struct MaterialProperties {
    std::string name;
    double density;           // g/cm^3
    double hydrogen_content;   // wt%
    double z_effective;       // Effective atomic number
    double radiation_length;   // g/cm^2
    double nuclear_interaction_length; // g/cm^2

    // Radiation attenuation
    double gcr_proton_reduction; // % at 10 g/cm^2
    double spe_proton_attenuation; // Factor at 5 g/cm^2

    // Physics parameters
    double displacement_energy; // eV
    double diffusion_coefficient; // m^2/s
    double migration_energy; // eV
    std::vector<double> defect_formation_energies; // eV

    // NASA model parameters
    double yield_strength; // MPa
    double vacuum_modifier;
    double ao_modifier; // Atomic oxygen
    double radiation_tolerance; // 0-100 scale

    double calculateThresholdForTemperature(double temperature_K) const;
};
```

17.9 Unified API

File: api/rad_ml.hpp

Single-header access to all framework components:

```
namespace rad_ml {
    // Framework version
    struct Version {
        static constexpr int major = 2;
        static constexpr int minor = 0;
        static constexpr int patch = 0;
    };

    // Initialize/shutdown
    bool initialize(bool enable_logging = true,
                   memory::MemoryProtectionLevel level = NONE);
    bool shutdown(bool check_for_leaks = true);

    // TMR factory functions
    namespace make_tmr {
        template <typename T> auto standard(const T& value);
        template <typename T> auto enhanced(const T& value);
        template <typename T> auto stuckBit(const T& value);
        template <typename T> auto healthWeighted(const T& value);
        template <typename T> auto approximate(const T& value, T tolerance);
        template <typename T> auto hybrid(const T& value);
    }

    // Memory management
    namespace memory_management {
        template <typename T>
```

```

    T* allocate(size_t count, MemoryProtectionLevel level);
    bool deallocate(void* ptr);
    size_t checkForLeaks(bool report = true);
}

// Simulation
namespace simulation {
    auto createRadiationSimulator(Environment env, double intensity);
    auto createMissionSimulator(const std::string& type, size_t days);
    auto createFaultInjector(double fault_rate);
}
}

```

17.10 Framework Component Summary

Table 31: Complete RadML Component Inventory

Module	Key Files	Purpose
api/	rad_ml.hpp	Unified API entry point
advanced/	algorithmic_diversity.hpp	Multi-algorithm voting
	error_prediction.hpp	ML-based error detection
core/	material_database.hpp	Material properties
	memory_scrubber.hpp	Periodic error correction
	checkpoint_manager.hpp	State recovery
	error_tracker.hpp	Statistics tracking
neural/	adaptive_protection.hpp	Dynamic ECC selection
	galois_field.hpp	GF(256) arithmetic
	advanced_reed_solomon.hpp	RS encoding/decoding
	selective_hardening.hpp	Critical layer protection
	protected_neural_network.hpp	Full NN protection
physics/	radiation_physics.hpp	Weibull, Bendel, SEU
	quantum_field_theory.hpp	QFT framework
	quantum_enhanced_radiation.hpp	Quantum corrections
	transport_equation.hpp	Boltzmann transport
research/	auto_arch_search.hpp	Architecture optimization
	variational_autoencoder.hpp	VAE for compression
	radiation_aware_training.hpp	Hardened training
sim/	physics_radiation_simulator.hpp	Environment modeling
	mission_environment.hpp	Mission profiles
storage/	ai_native_database.hpp	VAE-compressed storage
testing/	mission_simulator.hpp	Full mission simulation
	fault_injector.hpp	Controlled fault injection
tmr/	tmr.hpp, enhanced_tmr.hpp	8 TMR variants across 10 headers
	physics_driven_protection.hpp	Environment-aware TMR
pytorch/	pytorch_integration.hpp	PyTorch bridge
validation/	nasa_es_a_validation_protocol.hpp	Standard compliance

References

- [1] NASA, “Mitigating In-Space Charging Effects—A Guideline,” NASA-HDBK-4002A, 2011.
- [2] A. J. Tylka et al., “CREME96: A Revision of the Cosmic Ray Effects on Micro-Electronics Code,” *IEEE Trans. Nucl. Sci.*, vol. 44, no. 6, pp. 2150–2160, 1997.
- [3] D. M. Sawyer and J. I. Vette, “AP-8 Trapped Proton Environment for Solar Maximum and Solar Minimum,” NASA/GSFC NSSDC 76-06, 1976.
- [4] J. I. Vette, “The AE-8 Trapped Electron Model Environment,” NASA/GSFC NSSDC 91-24, 1991.
- [5] E. L. Petersen et al., “Rate Prediction for Single Event Effects—A Critique,” *IEEE Trans. Nucl. Sci.*, vol. 39, no. 6, pp. 1577–1599, 1992.
- [6] W. J. Stapor et al., “Two Parameter Bendel Model Calculations for Predicting Proton Induced Upset,” *IEEE Trans. Nucl. Sci.*, vol. 37, no. 6, pp. 1966–1973, 1990.
- [7] I. S. Reed and G. Solomon, “Polynomial Codes Over Certain Finite Fields,” *J. Soc. Ind. Appl. Math.*, vol. 8, no. 2, pp. 300–304, 1960.
- [8] E. R. Berlekamp, “Nonbinary BCH Decoding,” in *Algebraic Coding Theory*, McGraw-Hill, 1968, ch. 10.
- [9] J. L. Massey, “Shift-Register Synthesis and BCH Decoding,” *IEEE Trans. Inf. Theory*, vol. IT-15, no. 1, pp. 122–127, 1969.
- [10] ECSS, “Methods for the Calculation of Radiation Received and its Effects, and a Policy for Design Margins,” ECSS-E-ST-10-12C, 2008.
- [11] JEDEC, “Test Procedures for the Measurement of Single-Event Effects in Semiconductor Devices from Heavy Ion Irradiation,” JESD57A, 2017.
- [12] R. A. Reed et al., “Anthology of the Development of Radiation Transport Tools as Applied to Single Event Effects,” *IEEE Trans. Nucl. Sci.*, vol. 60, no. 3, pp. 1876–1911, 2013.
- [13] NASA, “Single Event Effects Criticality Analysis,” NASA/TP-2006-214373, 2006.

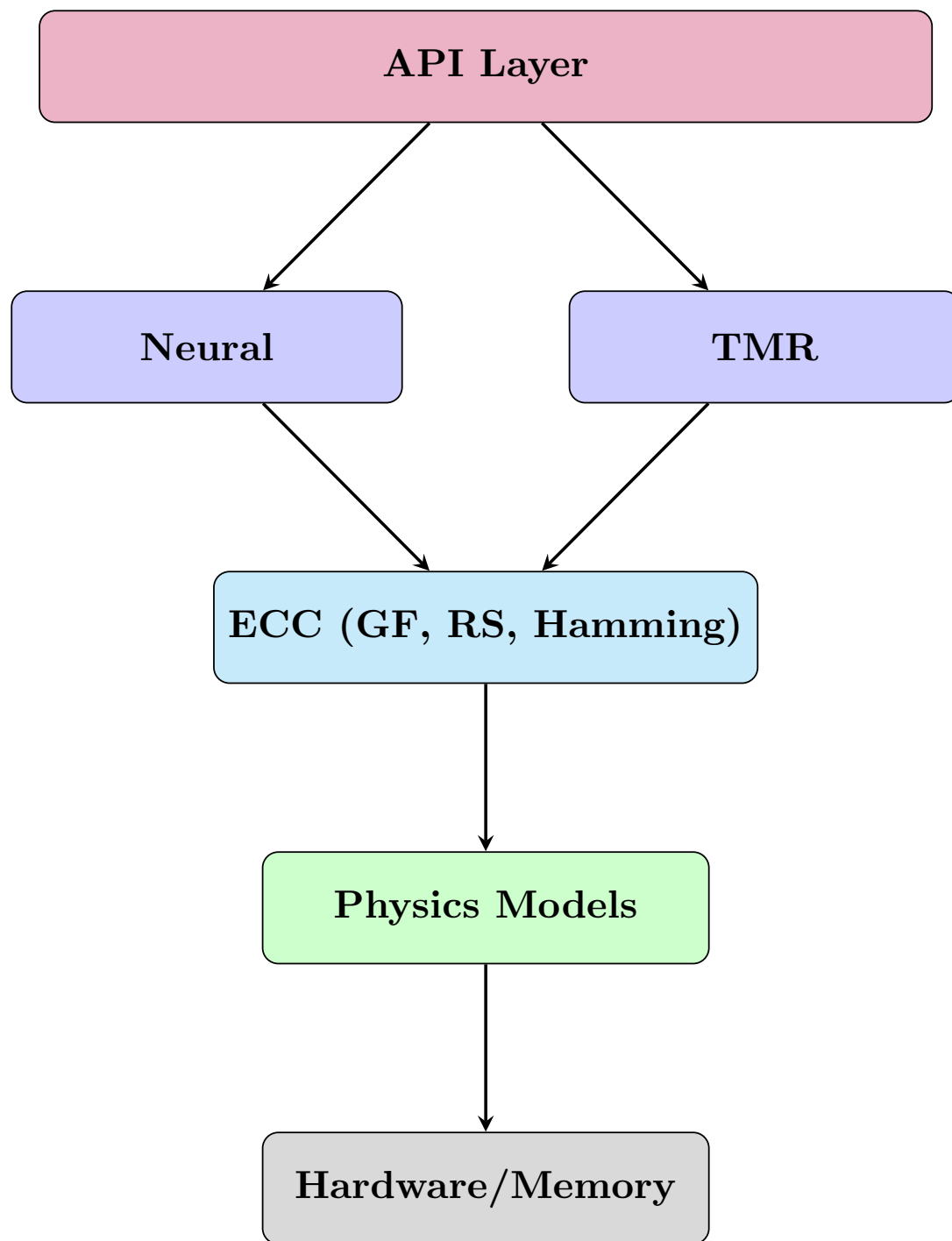


Figure 1: RadML System Architecture

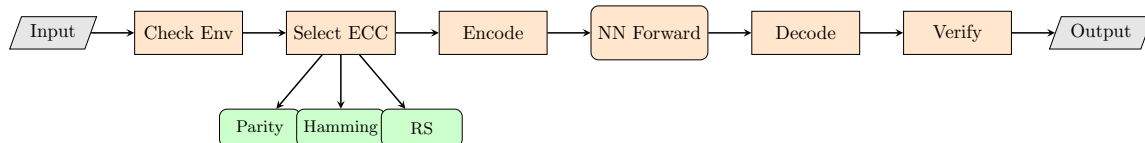


Figure 2: Protection Pipeline Flow

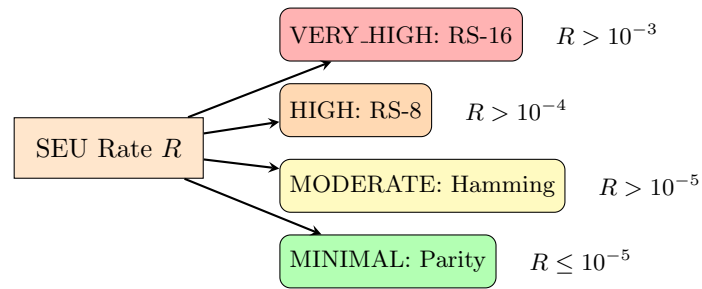


Figure 3: Adaptive Protection Level Selection

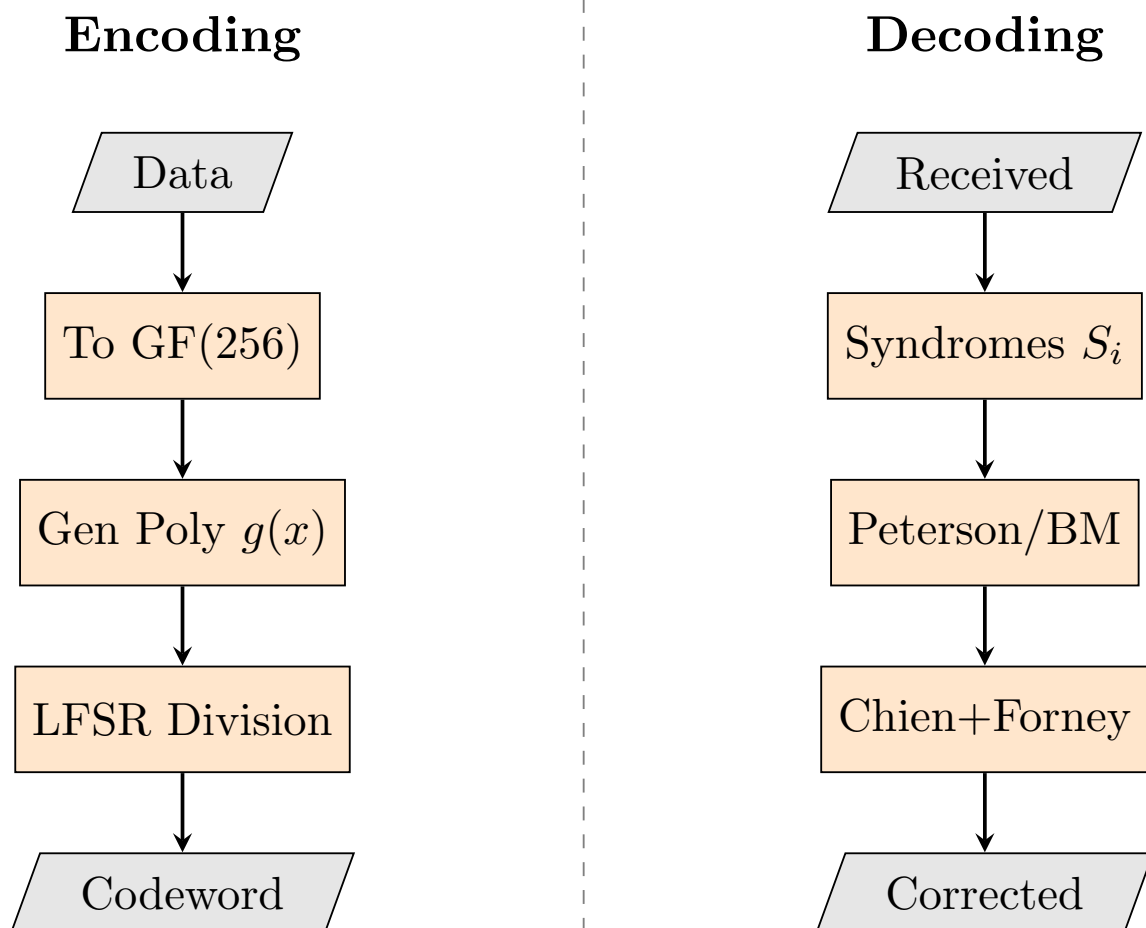


Figure 4: Reed-Solomon Encoding/Decoding

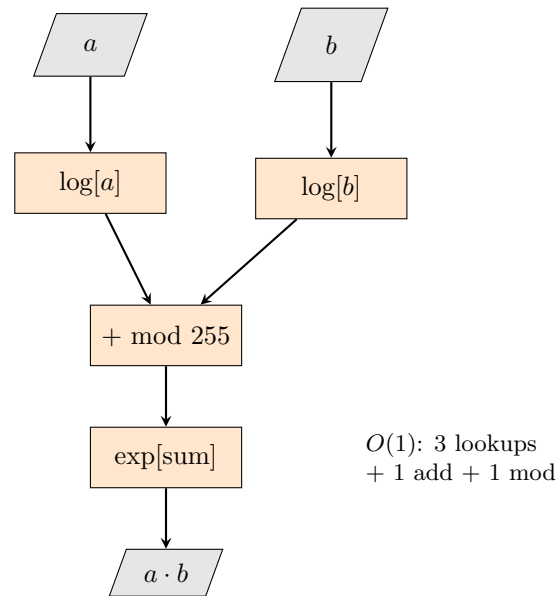


Figure 5: GF(256) Multiplication via Log Tables

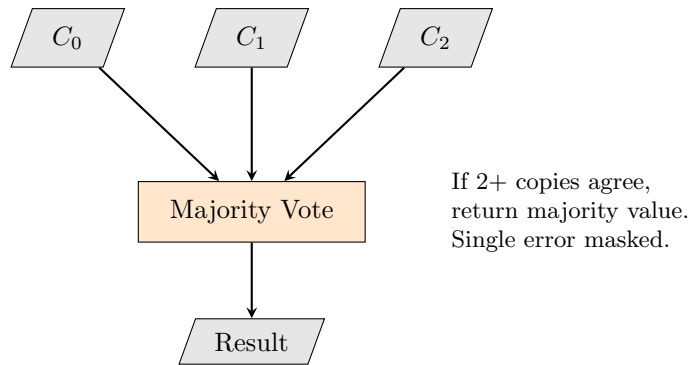


Figure 6: TMR Majority Voting

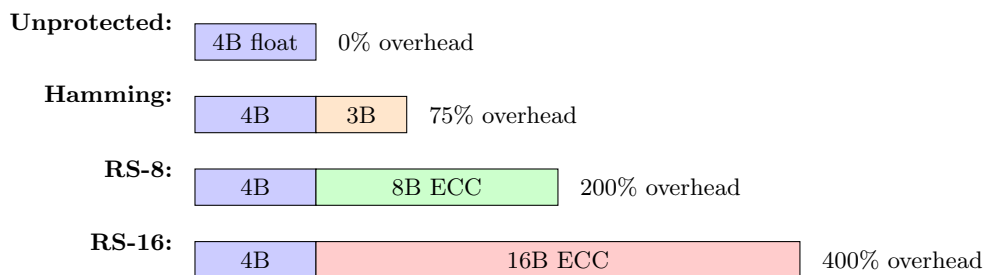


Figure 7: Memory Layout for Different Protection Levels

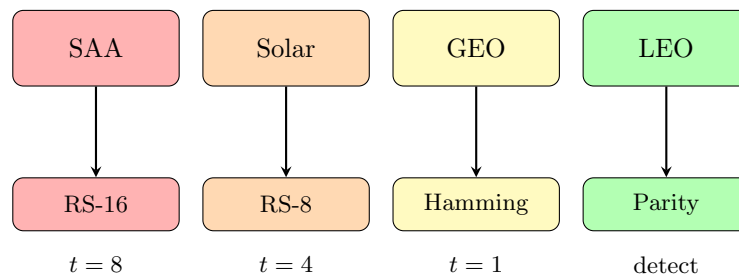


Figure 8: Radiation Environment to Protection Mapping

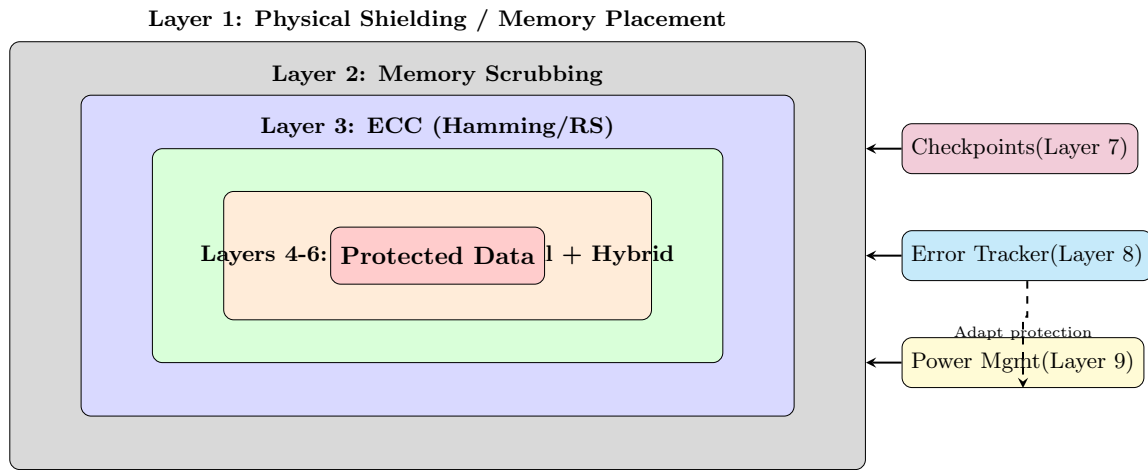


Figure 9: Defense-in-Depth: Nested Protection Layers

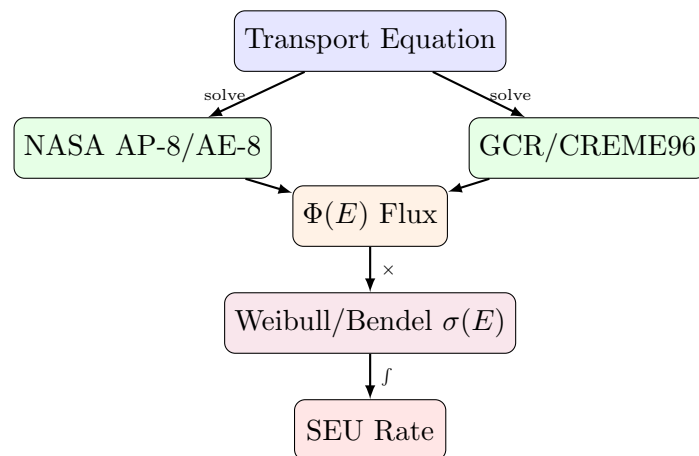


Figure 10: Transport Equation to SEU Rate Pipeline